



University
of Exeter

COMM113

Deep Learning – Week 9

Transformer and Language Models I

Gaojie (Jay) Jin
g.jin@exeter.ac.uk

Encoder

Encoder layers (repeated N times)

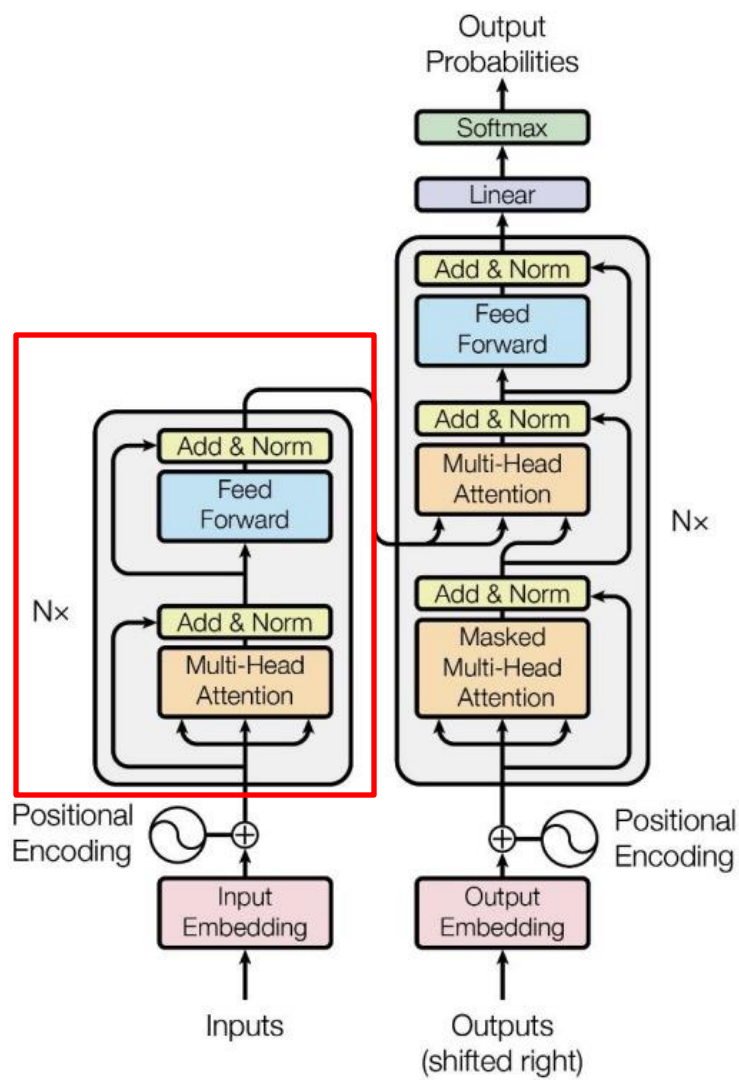
Each layer contains:

- Multi-Head (Self-) Attention
- Feed-Forward Network
- Add & Norm (Residual + LayerNorm)

Input Embedding + Positional Encoding

Output passed to the decoder later

Q: Is the Encoder responsible for prediction?



Encoder

Encoder layers (repeated N times)

Each layer contains:

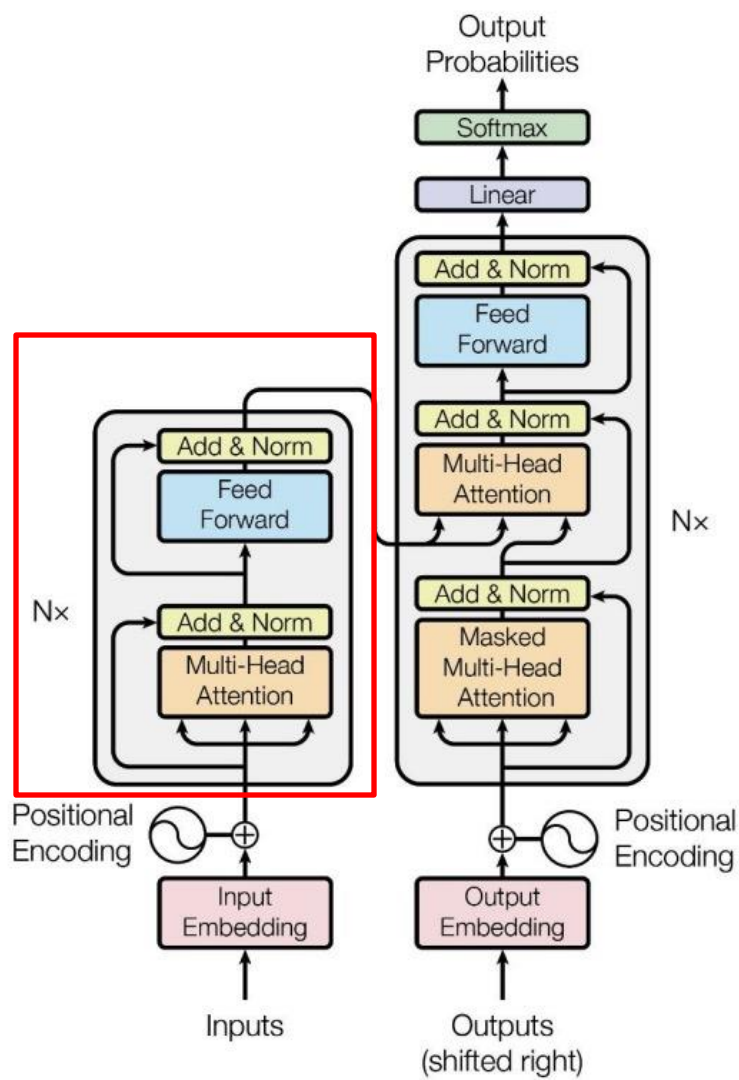
- Multi-Head (Self-) Attention
- Feed-Forward Network
- Add & Norm (Residual + LayerNorm)

Input Embedding + Positional Encoding

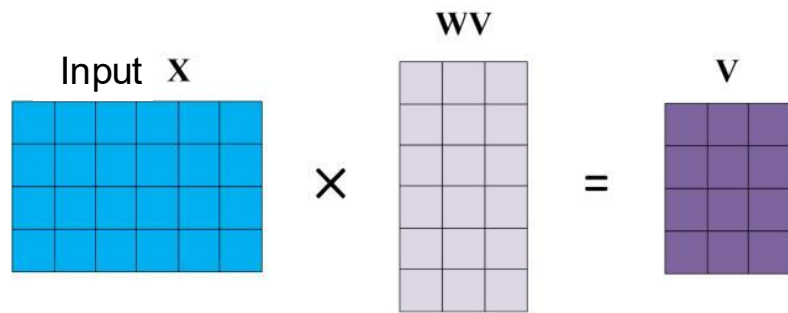
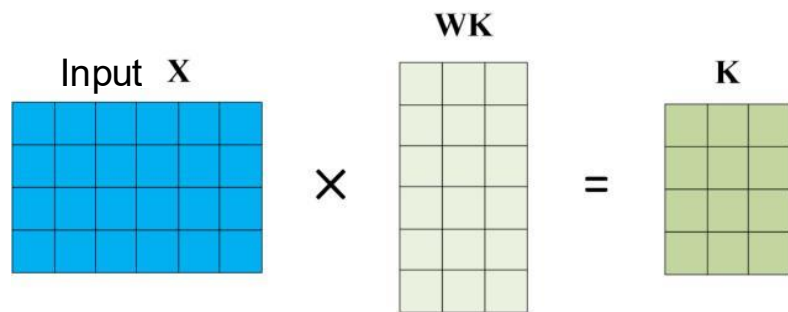
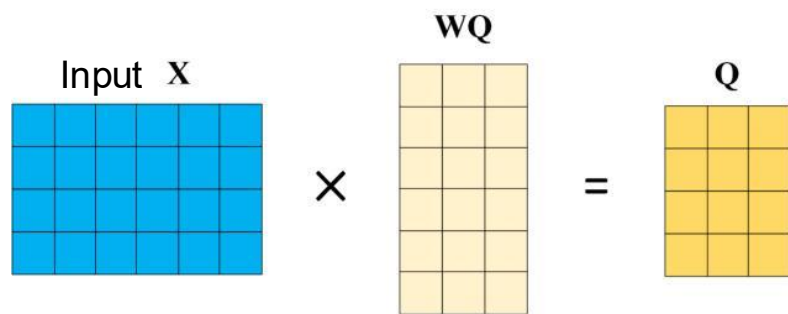
Output passed to the decoder later

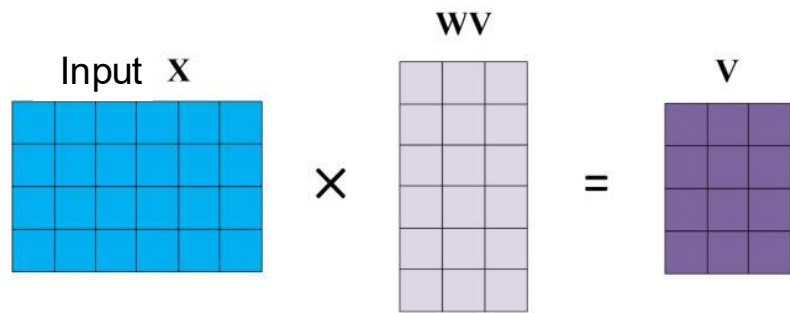
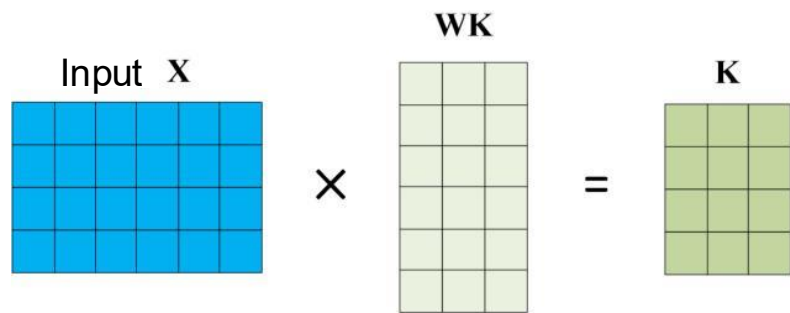
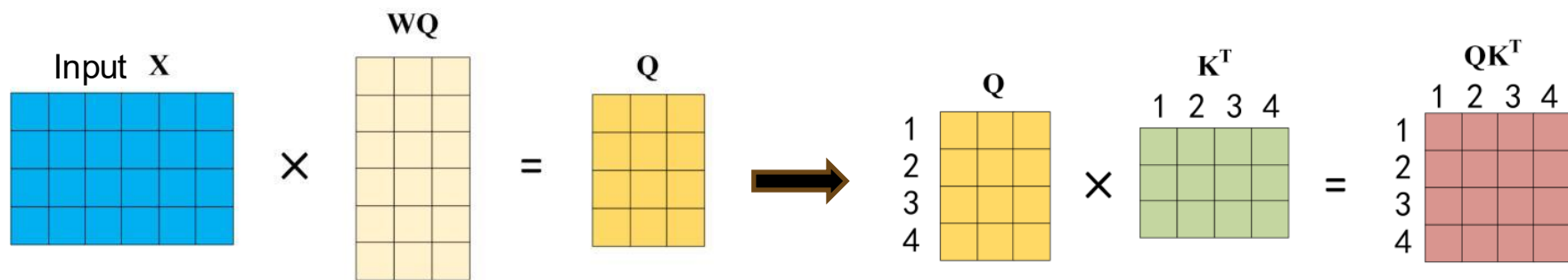
Q: Is the Encoder responsible for prediction?

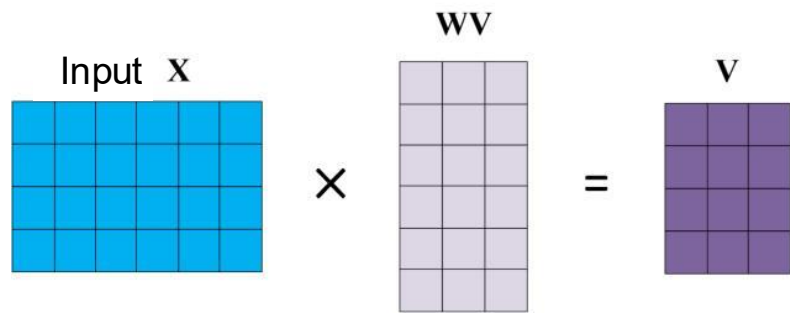
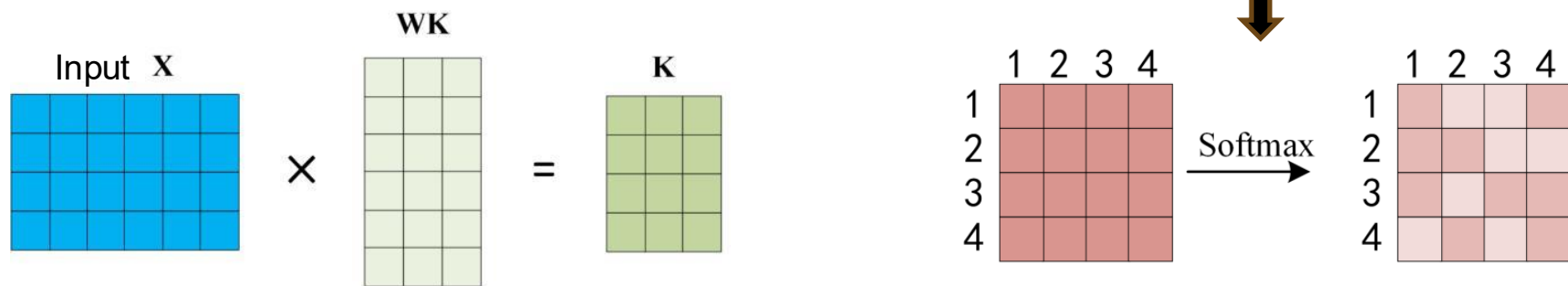
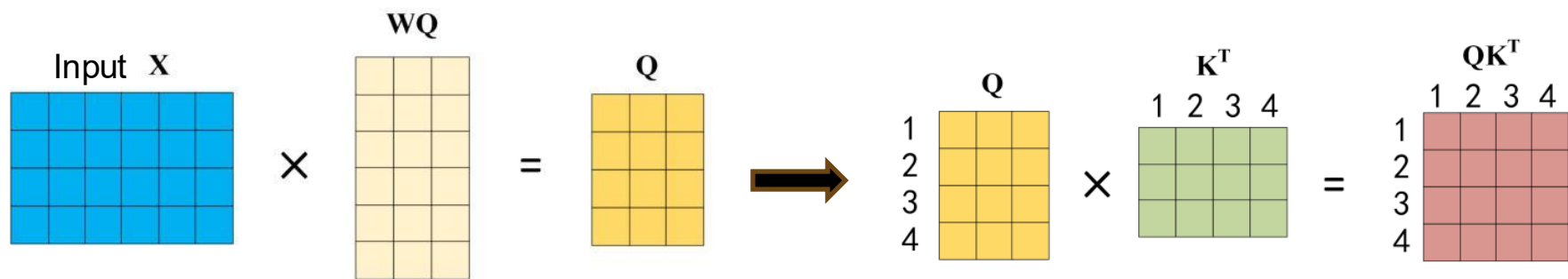
A: No, only representation.

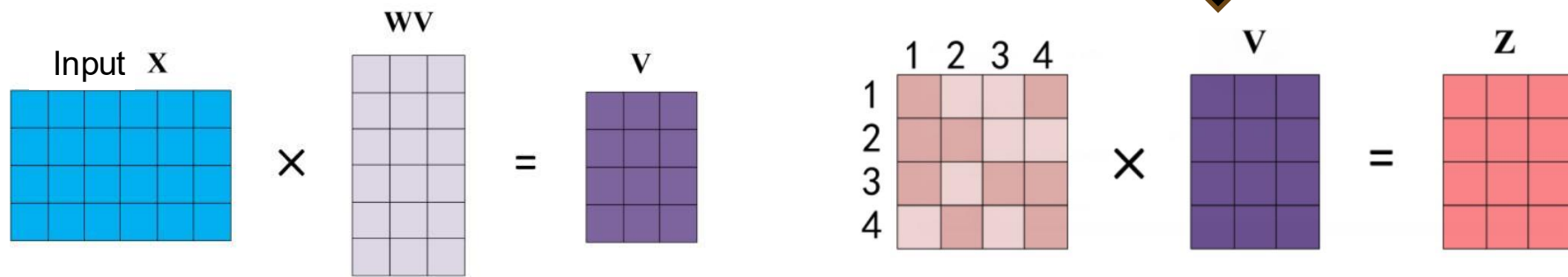
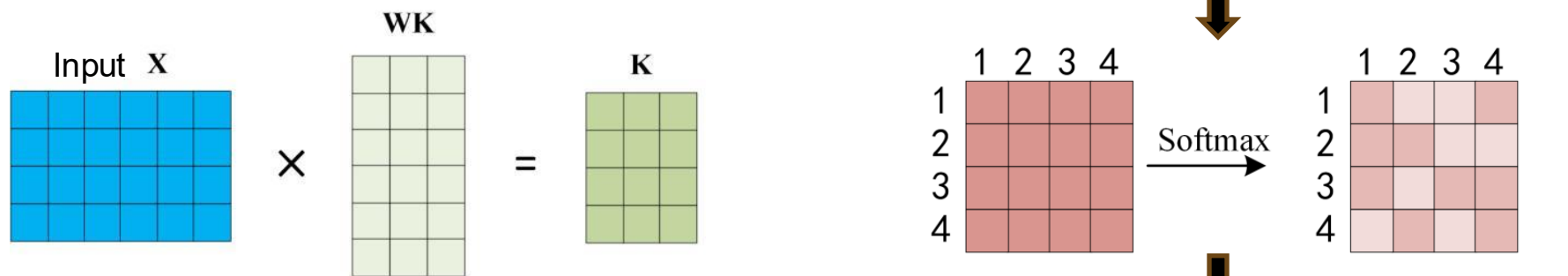
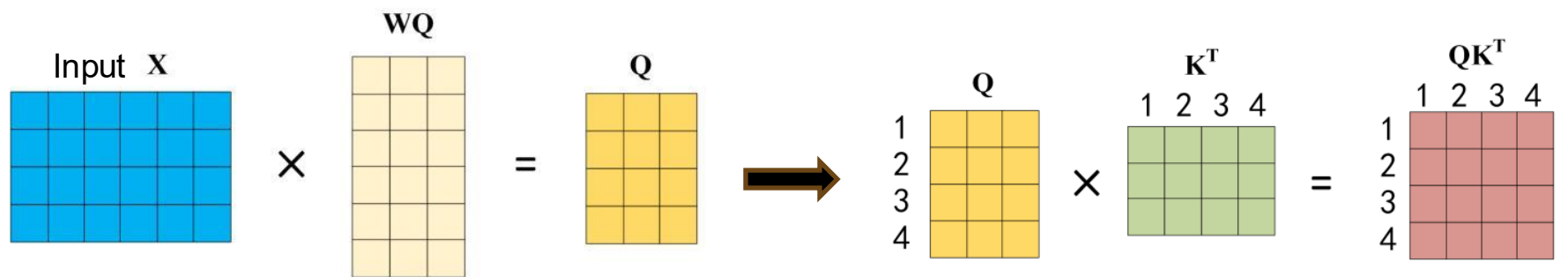


Self-Attention









Decoder

Decoder layers (repeated N times)

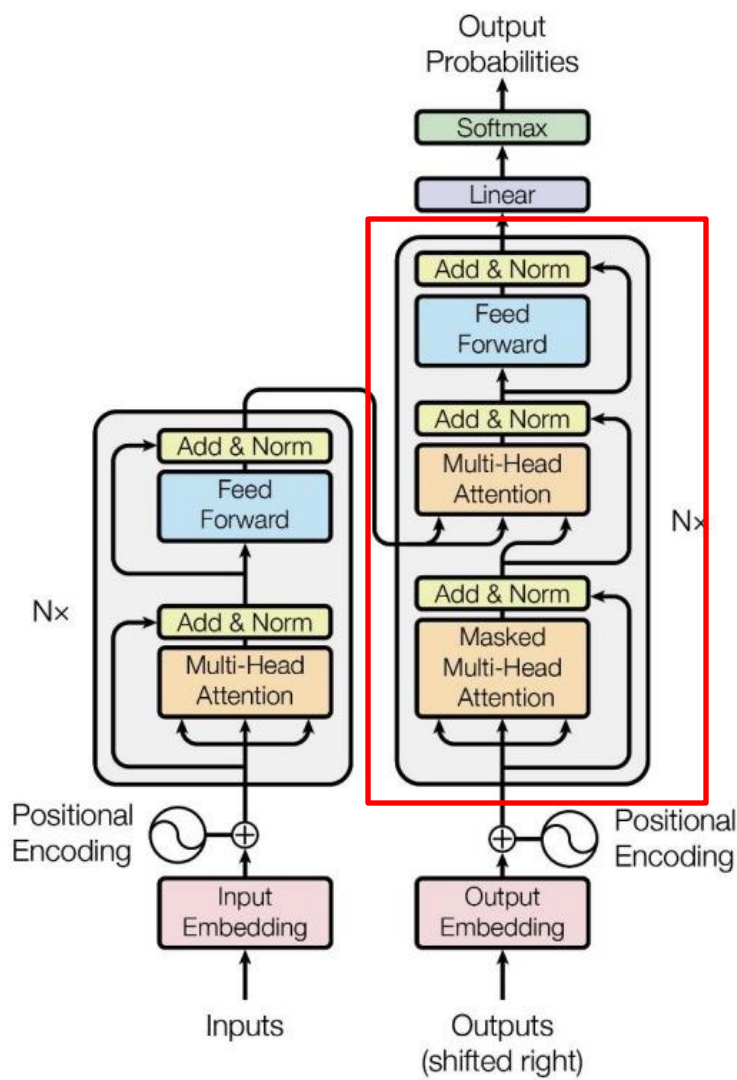
Three sublayers:

- Masked Multi-Head (Self-) Attention
- Encoder–Decoder Attention
- Feed Forward Network

Add & Norm after each block

Output probabilities

Q: Why masking?



Decoder

Decoder layers (repeated N times)

Three sublayers:

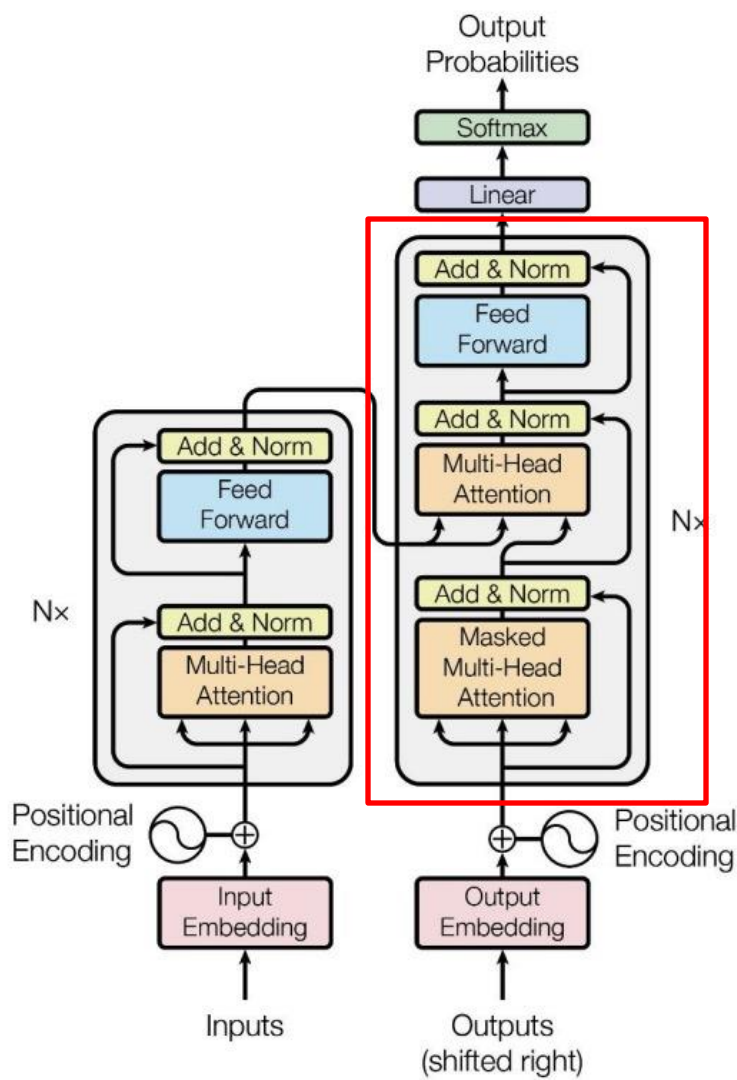
- Masked Multi-Head (Self-) Attention
- Encoder–Decoder Attention
- Feed Forward Network

Add & Norm after each block

Output probabilities

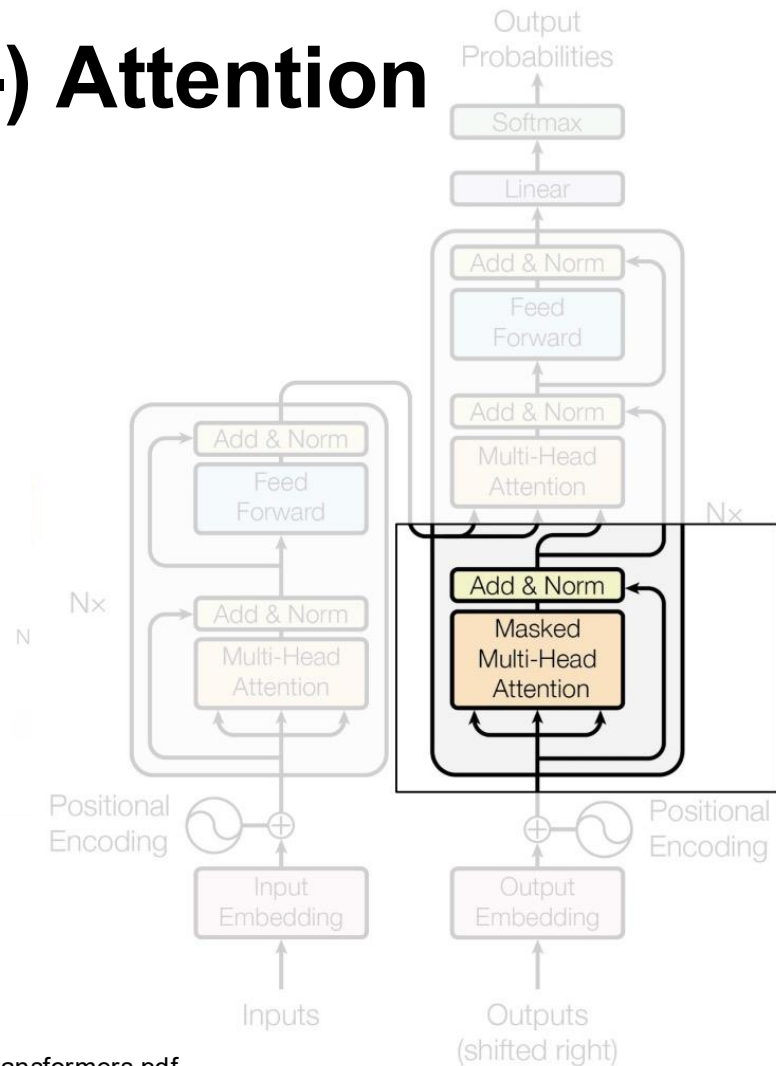
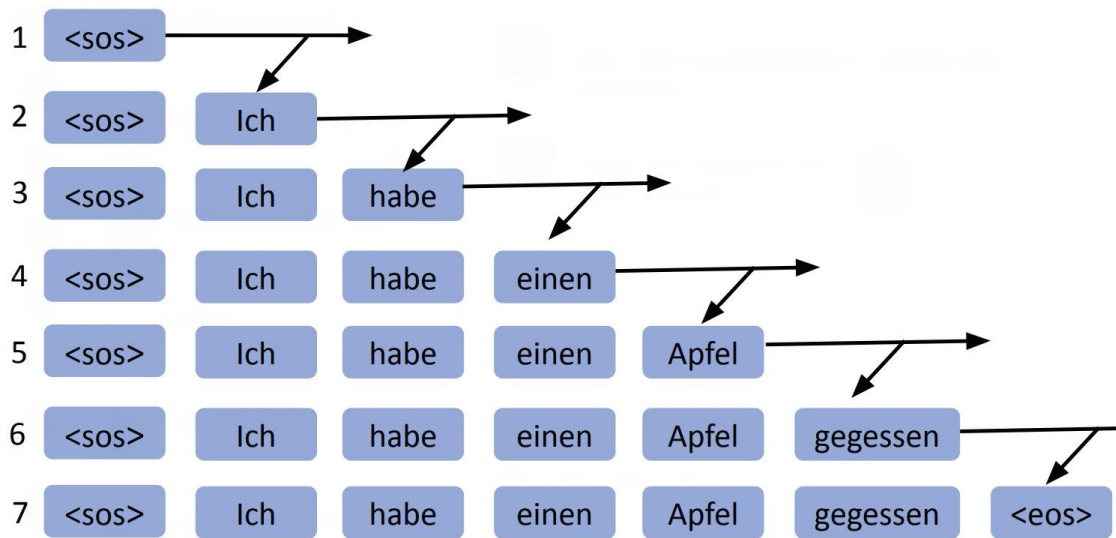
Q: Why masking?

A: Prevents seeing future words.



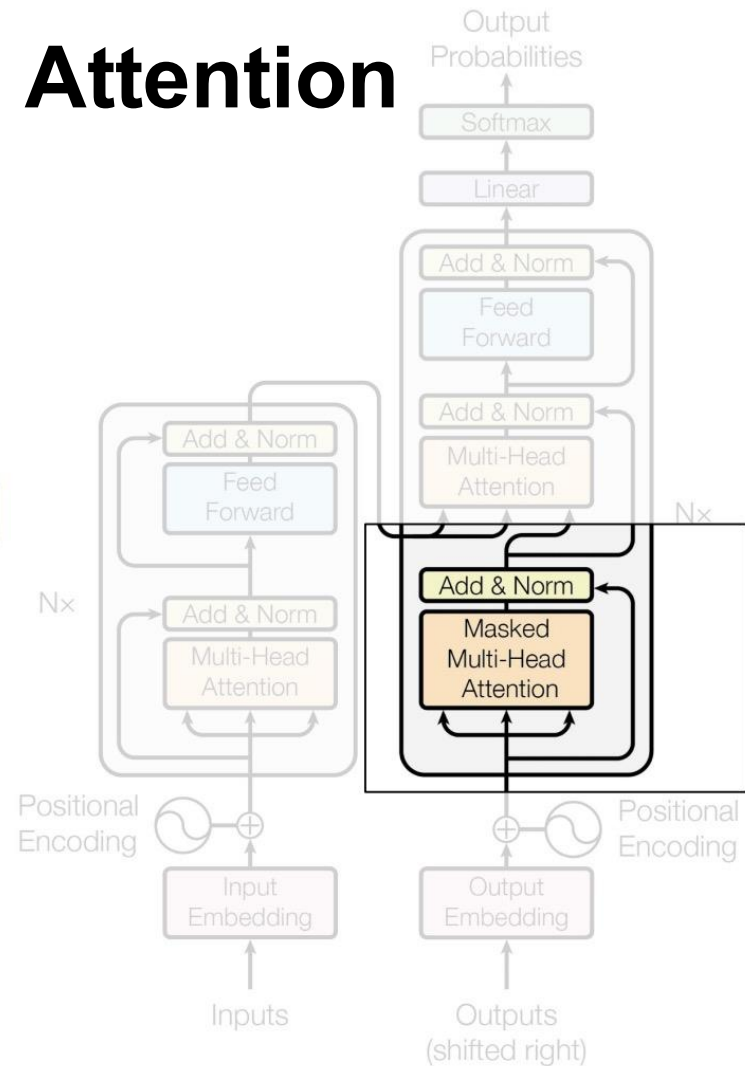
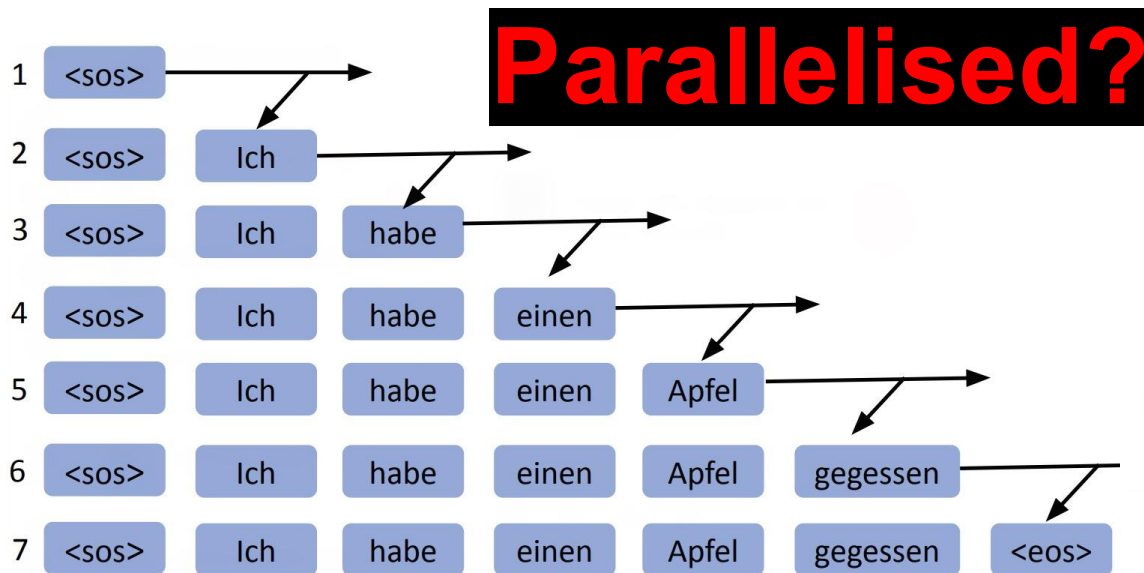
Masked Multi Head (or Self-) Attention

Decoding step by step (using Teacher Forcing)



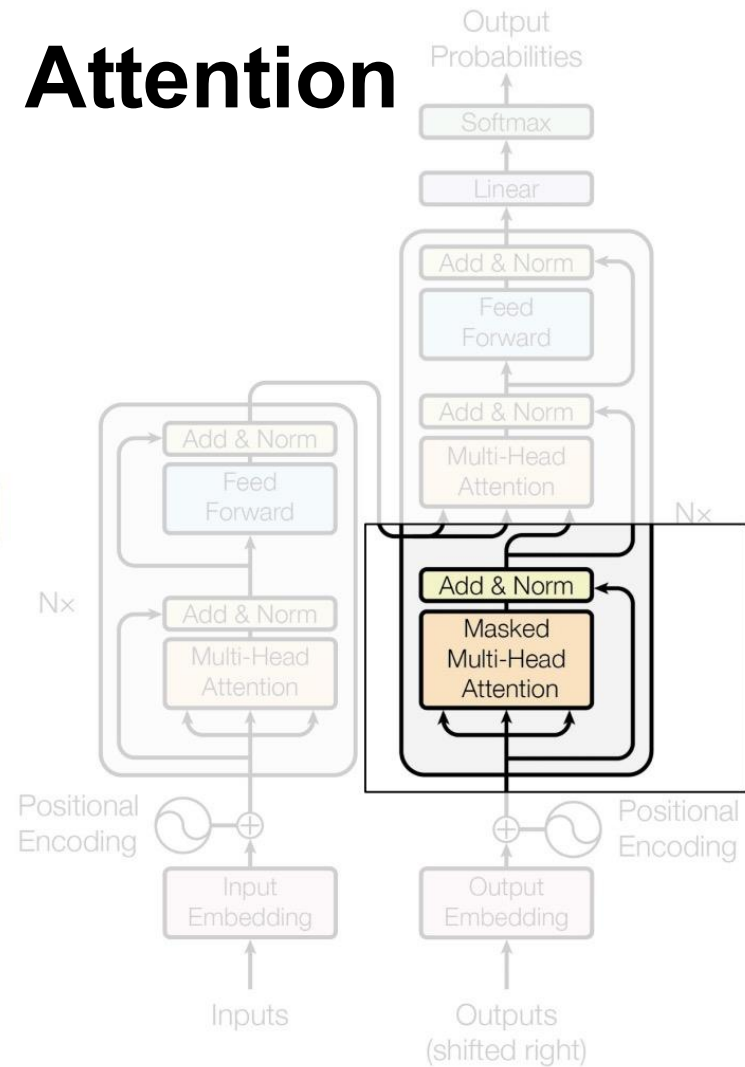
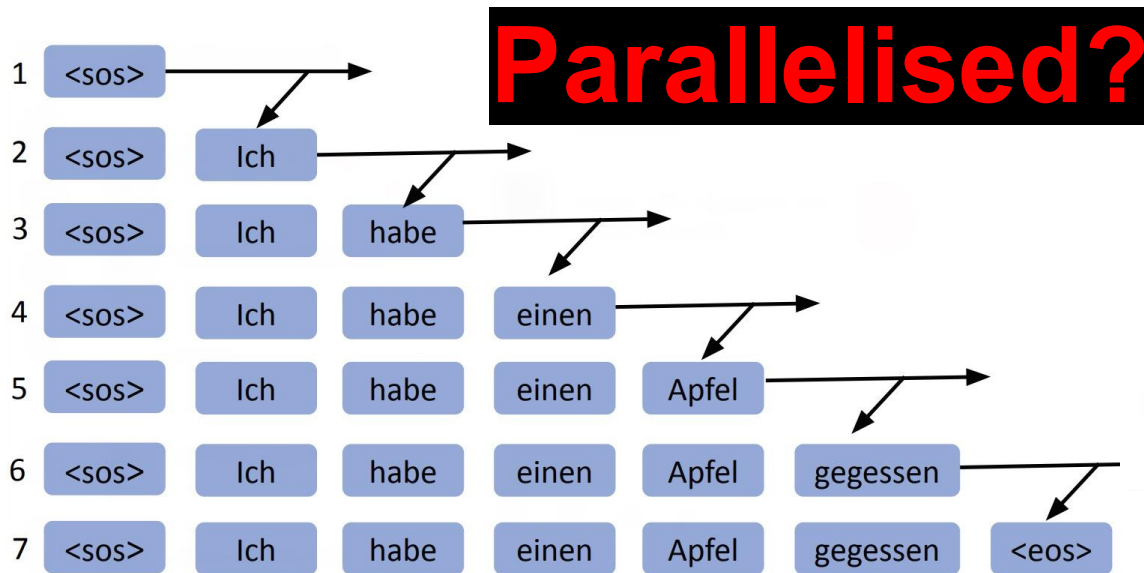
Masked Multi Head (or Self-) Attention

Decoding step by step (using Teacher Forcing)



Masked Multi Head (or Self-) Attention

Decoding step by step (using **Teacher Forcing**)



Teacher Forcing

- A training technique where the model is fed the correct, ground-truth token from the previous step as input, instead of its own prediction

For **example**, if the target sequence is "**I like eating mushrooms**,"

1. the model's decoder would receive "**I like**" as input for the first step,
2. "**I like eating**" as input for the second, and so on.

Problem:

Teacher Forcing

- A training technique where the model is fed the correct, ground-truth token from the previous step as input, instead of its own prediction

For **example**, if the target sequence is "**I like eating mushrooms**,"

1. the model's decoder would receive "**I like**" as input for the first step,
2. "**I like eating**" as input for the second, and so on.

Problem: Teacher forcing creates a mismatch between training and inference. During inference, the model must use its own previous prediction as the input for the next step.

This is known as **exposure bias**.

Techniques like **scheduled sampling** are used to mitigate this by gradually shifting from teacher forcing to using the model's own predictions during training.

Teacher Forcing

- A training technique where the model is fed the correct, ground-truth token from the previous step as input, instead of its own prediction

For **example**, if the target sequence is "**I like eating mushrooms**,"

1. the model's decoder would receive "**I like**" as input for the first step,
2. "**I like eating**" as input for the second, and so on.

Problem: Teacher forcing creates a mismatch between training and inference. During inference, the model must use its own previous prediction as the input for the next step.

Q: Why can't inference be parallel?

Teacher Forcing

- A training technique where the model is fed the correct, ground-truth token from the previous step as input, instead of its own prediction

For **example**, if the target sequence is "**I like eating mushrooms**,"

1. the model's decoder would receive "**I like**" as input for the first step,
2. "**I like eating**" as input for the second, and so on.

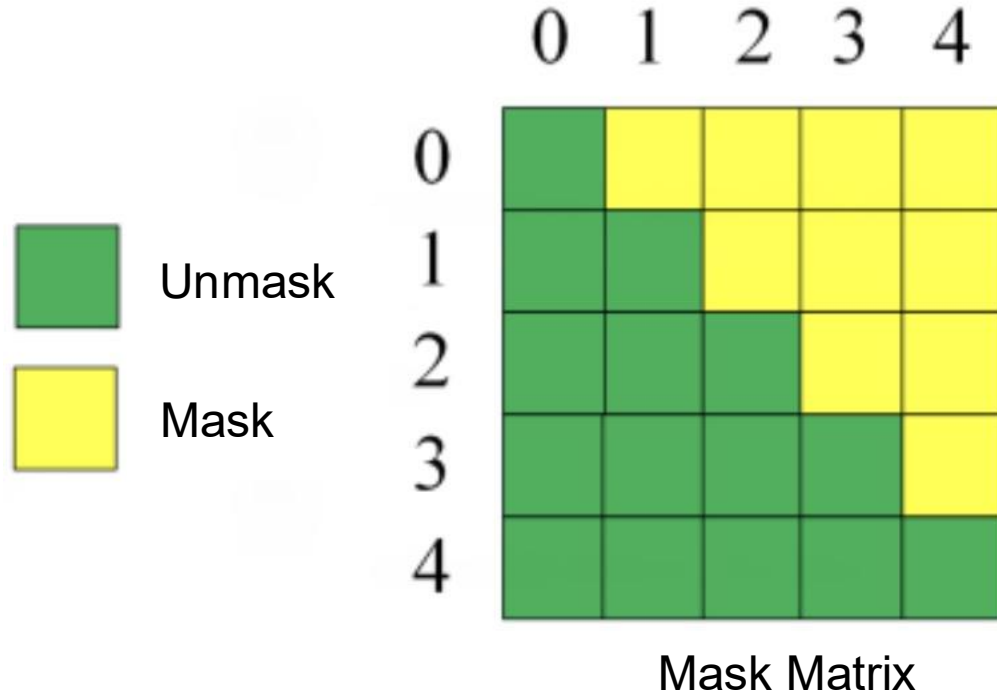
Problem: Teacher forcing creates a mismatch between training and inference. During inference, the model must use its own previous prediction as the input for the next step.

Q: Why can't inference be parallel?

A: Future tokens unknown.

Masked Multi Head (or Self-) Attention

First step:



Masked Multi Head (or Self-) Attention

Second step:

$$\begin{array}{c} \mathbf{Q} \\ \begin{array}{l} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \end{array} \times \begin{array}{c} \mathbf{K}^T \\ \begin{array}{l} 0 \quad 1 \quad 2 \quad 3 \quad 4 \end{array} \\ \begin{array}{l} \\ \\ \\ \\ \end{array} \end{array} = \begin{array}{c} \mathbf{QK}^T \\ \begin{array}{l} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \end{array} \begin{array}{c} \begin{array}{l} 0 \quad 1 \quad 2 \quad 3 \quad 4 \end{array} \\ \begin{array}{l} \\ \\ \\ \\ \end{array} \end{array}$$

The diagram illustrates the second step of the masked multi-head attention mechanism, which is the matrix multiplication of the Query matrix ($\mathbf{Q}$) and the Transposed Key matrix ($\mathbf{K}^T$) to produce the attention scores matrix ($\mathbf{QK}^T$).

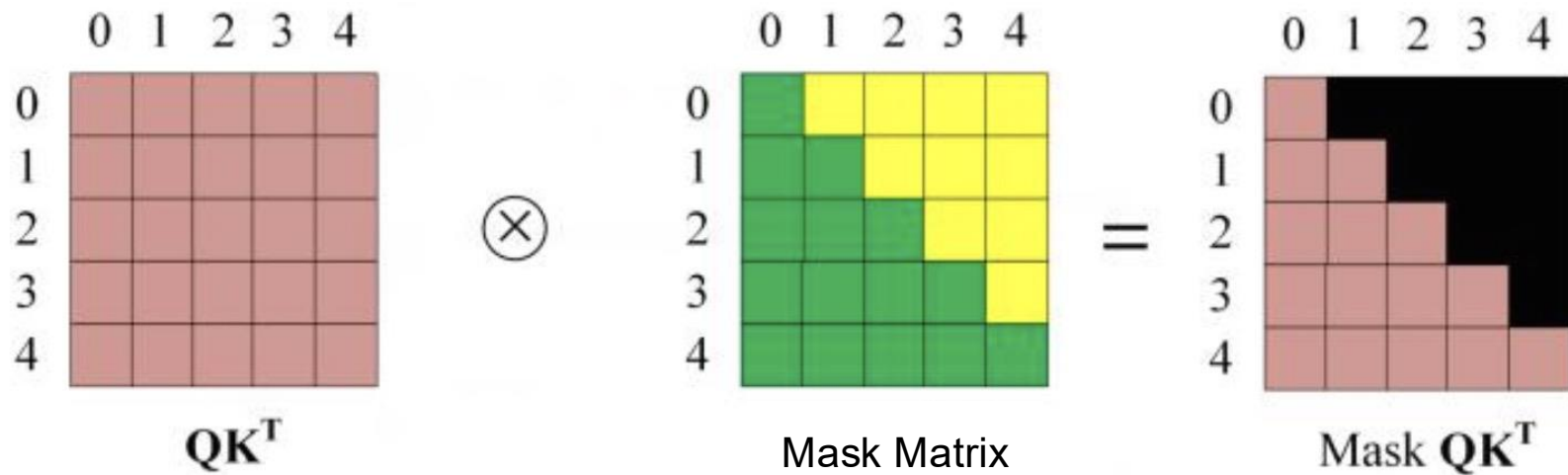
Matrix $\mathbf{Q}$ (Query): A 5x3 matrix (rows 0-4, columns 1-3) represented by yellow cells.

Matrix $\mathbf{K}^T$ (Transposed Key): A 5x5 matrix (rows 0-4, columns 0-4) represented by green cells.

Matrix $\mathbf{QK}^T$ (Attention Scores): A 5x5 matrix (rows 0-4, columns 0-4) represented by red cells.

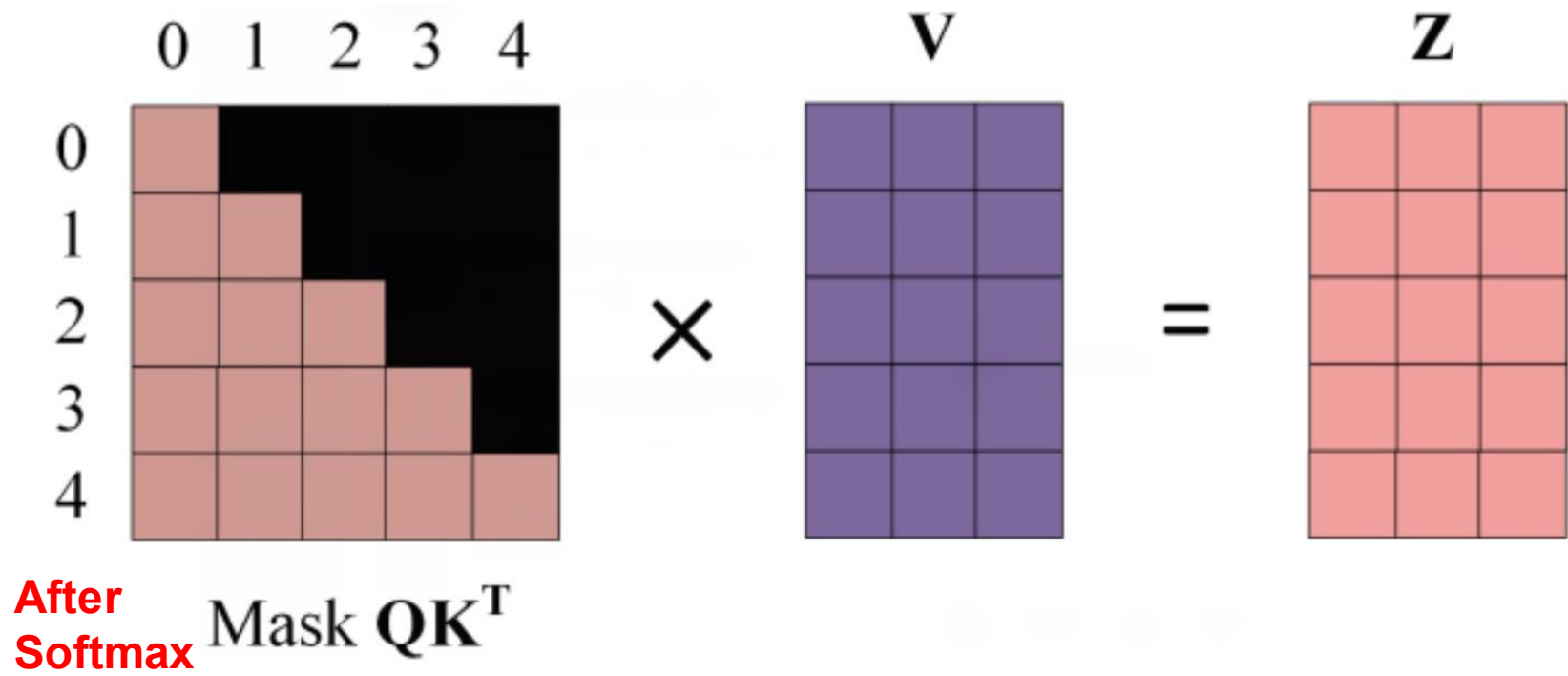
Masked Multi Head (or Self-) Attention

Third step:



Masked Multi Head (or Self-) Attention

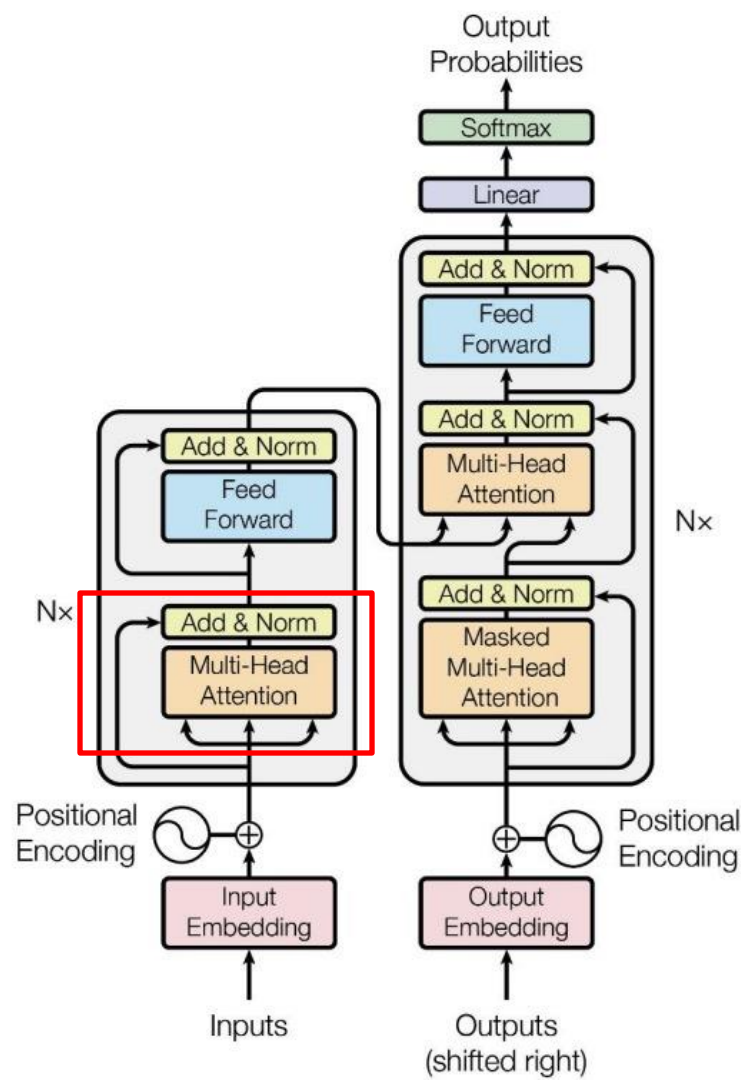
Fourth step:



Encoder–Decoder

Encoder Self Attention **Inputs:**

- Queries from Encoder Inputs
- Keys from Encoder Inputs
- Values from Encoder Inputs



Encoder–Decoder

Encoder Self Attention **Inputs:**

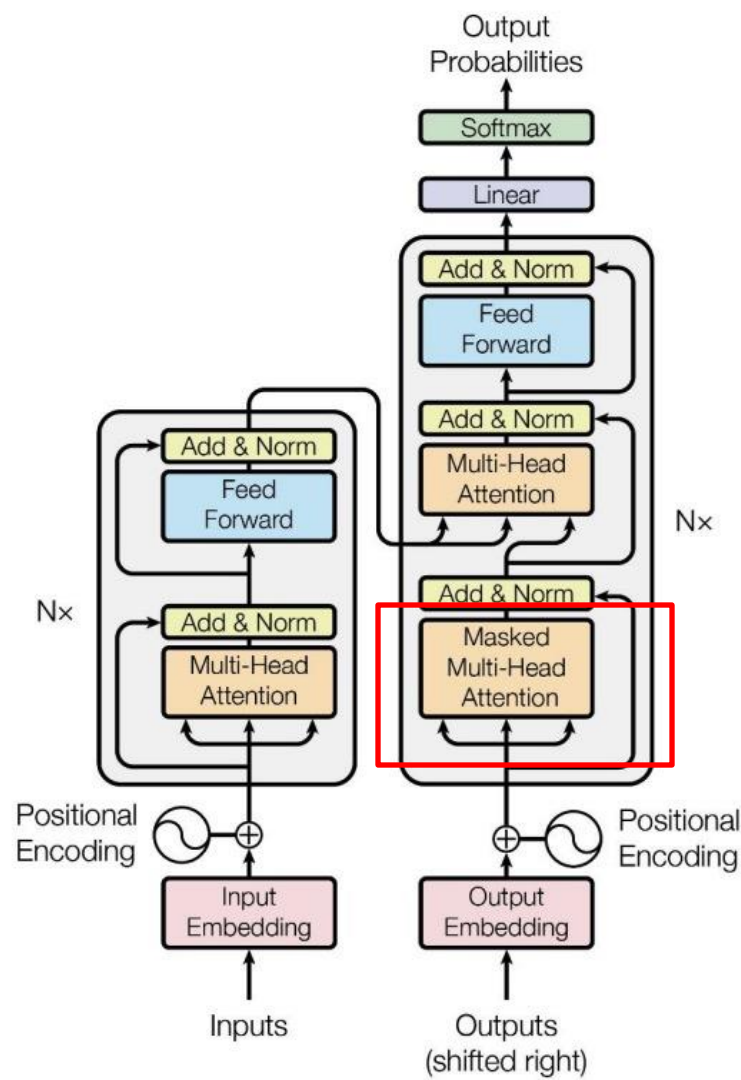
- Queries from Encoder Inputs
- Keys from Encoder Inputs
- Values from Encoder Inputs

Decoder Masked Self Attention **Inputs:**

- Queries from Decoder Inputs
- Keys from Decoder Inputs
- Values from Decoder Inputs

Decoder inputs: shifted-right embeddings for the first layer, or output of previous decoder layer

Q: What is shifted-right?



Encoder–Decoder

Encoder Self Attention **Inputs:**

- Queries from Encoder Inputs
- Keys from Encoder Inputs
- Values from Encoder Inputs

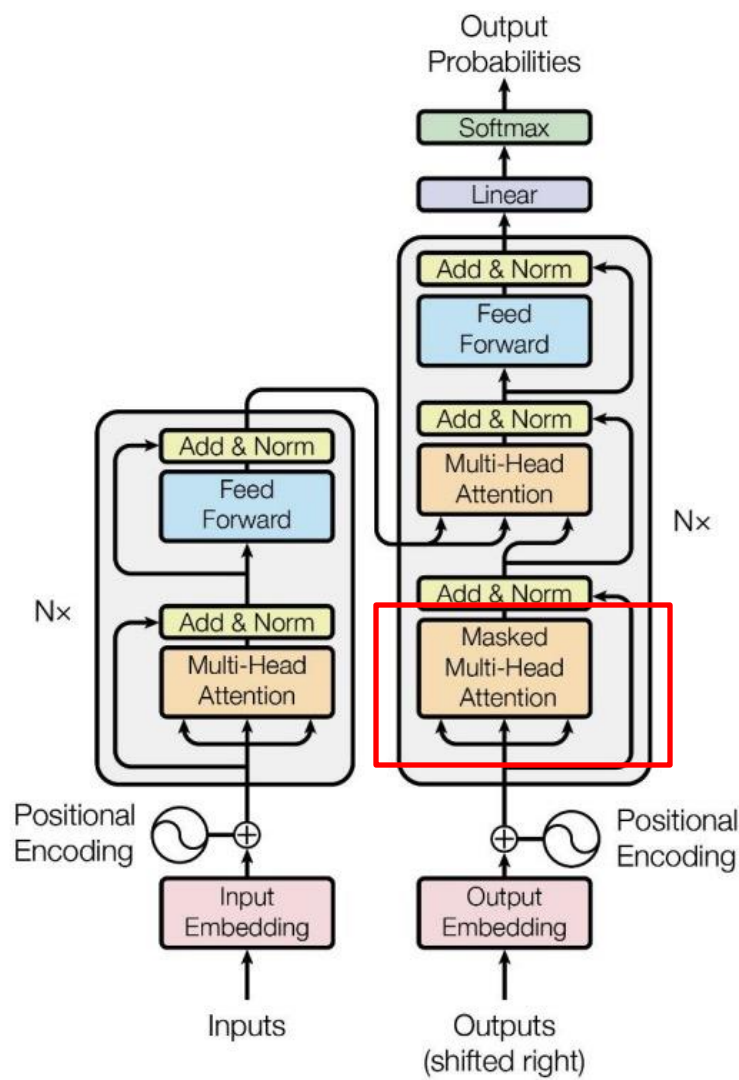
Decoder Masked Self Attention **Inputs:**

- Queries from Decoder Inputs
- Keys from Decoder Inputs
- Values from Decoder Inputs

Decoder inputs: shifted-right embeddings for the first layer, or output of previous decoder layer

Q: What is shifted-right?

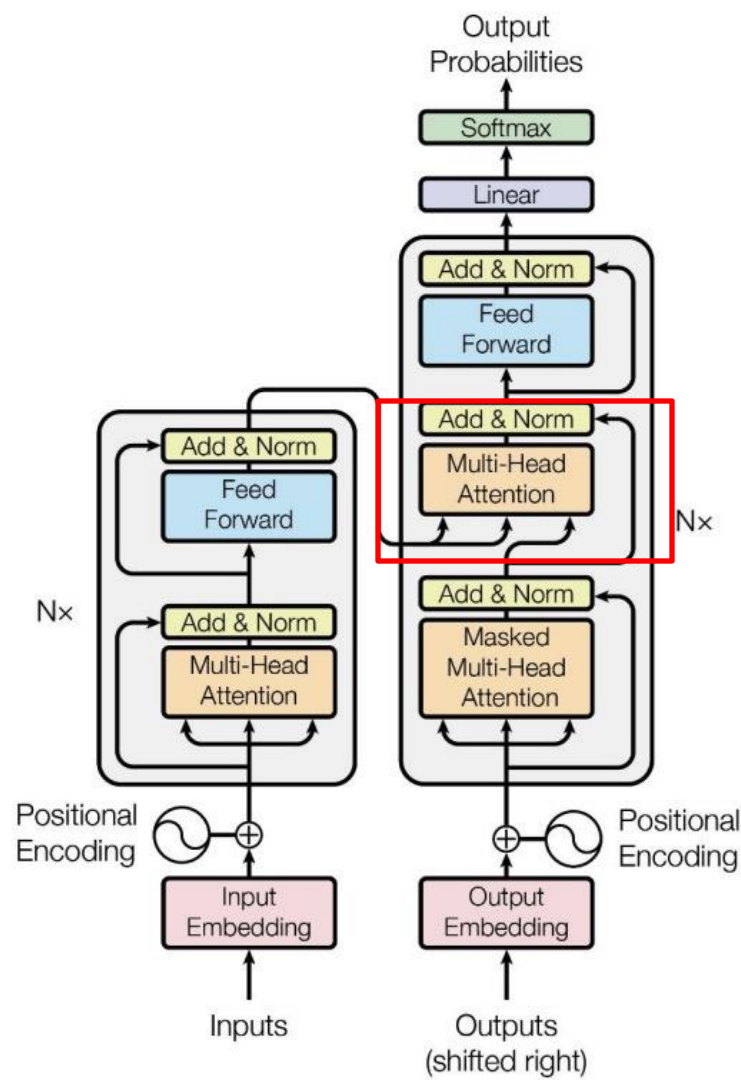
A: Input delayed one timestep.



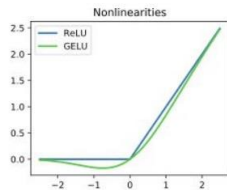
Encoder–Decoder

Encoder–Decoder Self Attention **Inputs:**

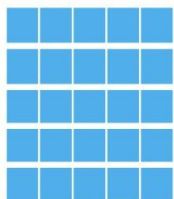
- Queries from Decoder Representation
- Keys from Encoder Outputs
- Values from Encoder outputs



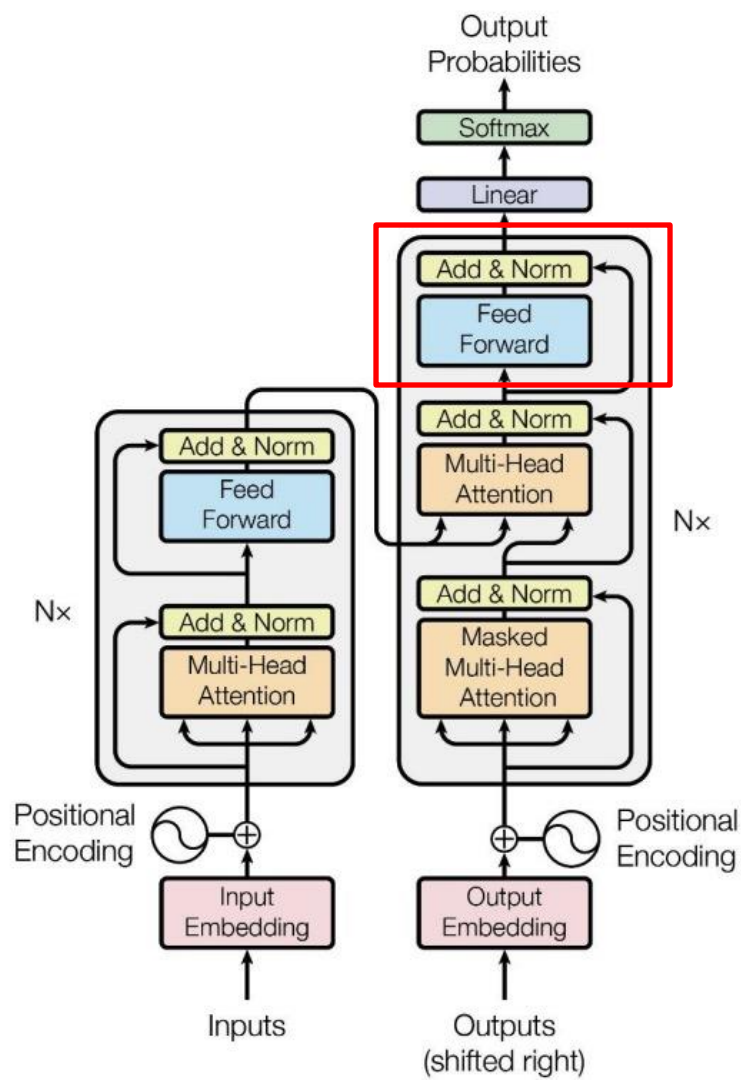
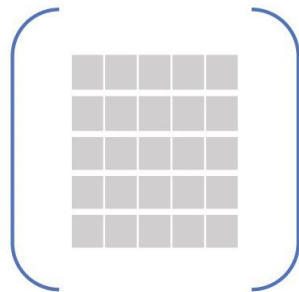
Decoder



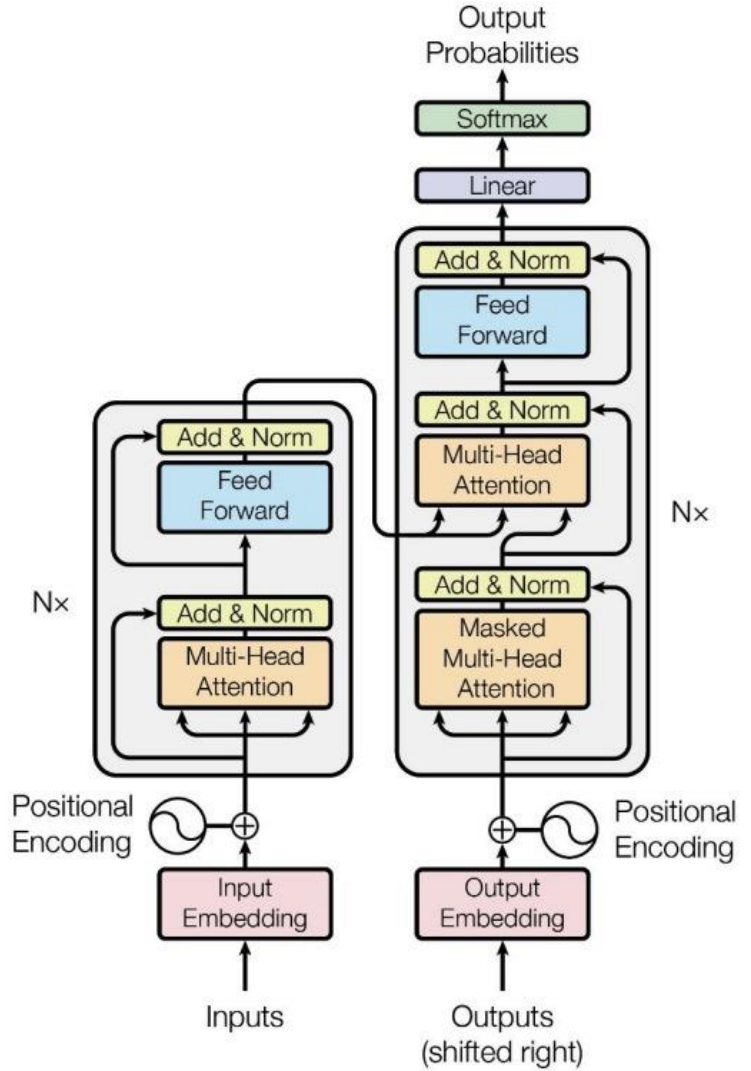
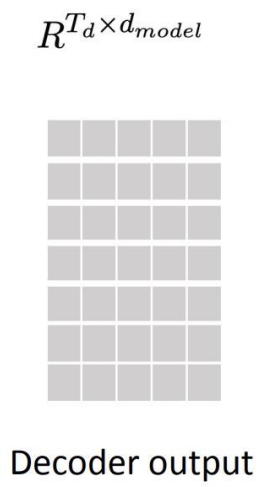
Feed Forward



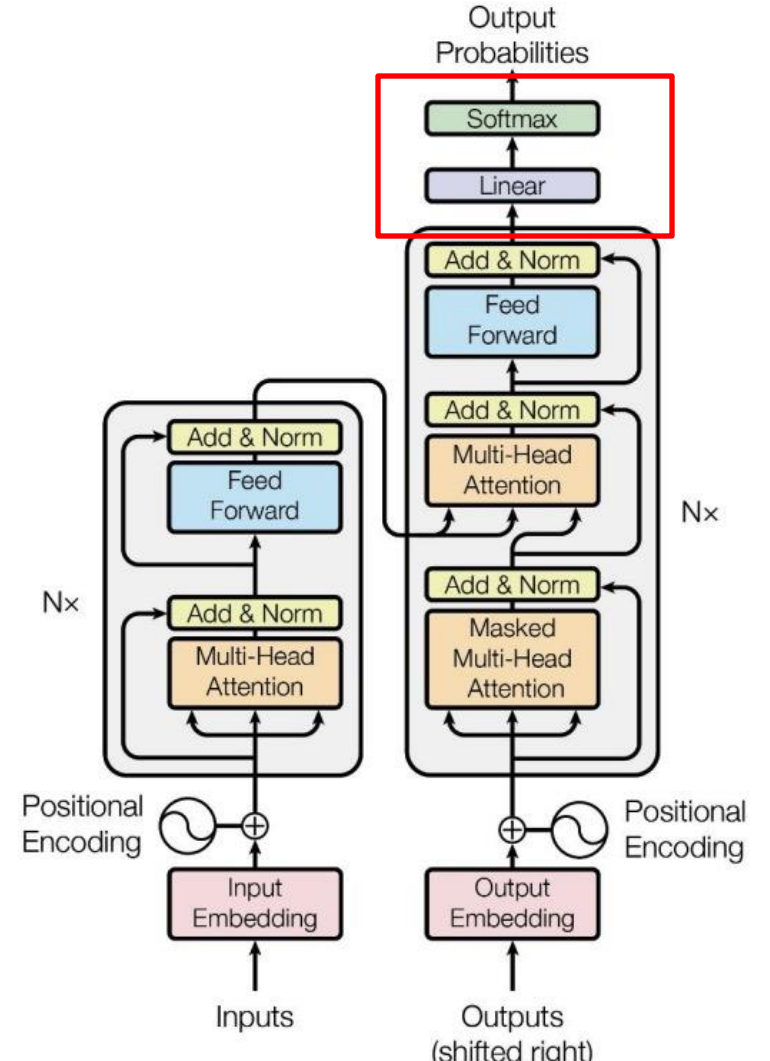
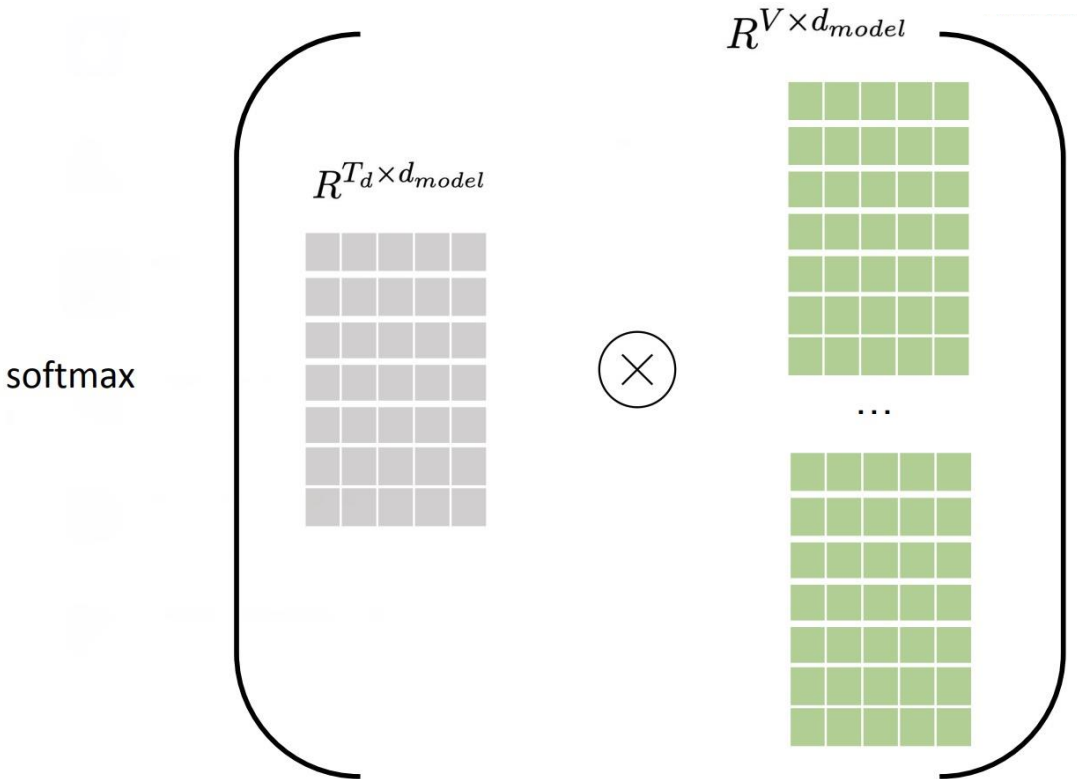
Residuals



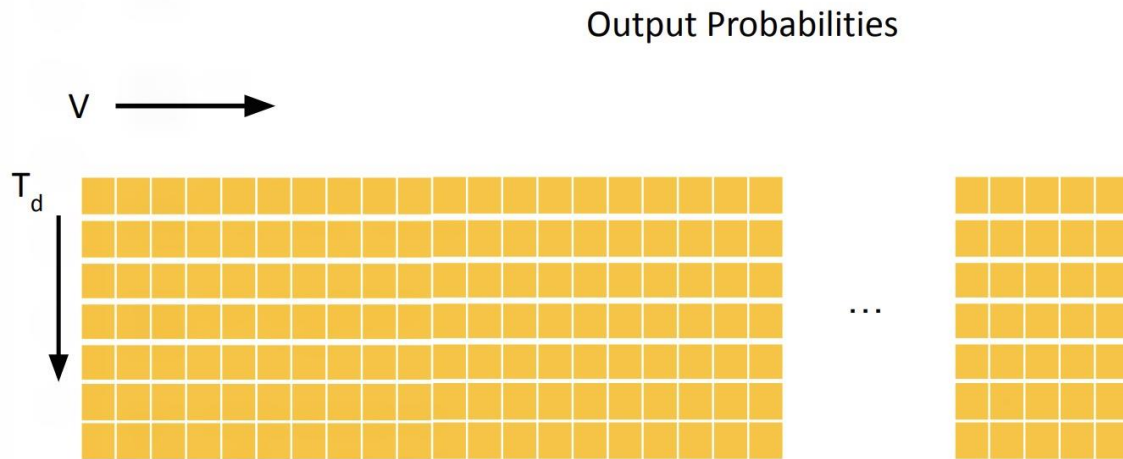
Decoder Block



Decoder - Linear



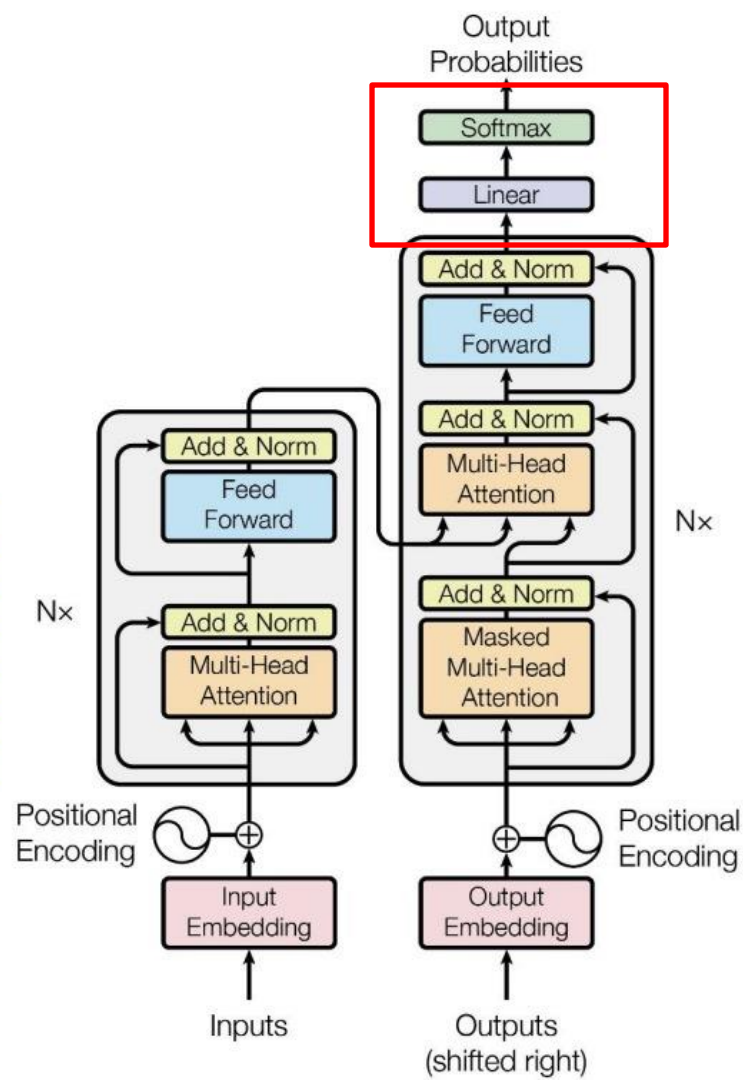
Decoder - Softmax



$$R^{T_d \times V}$$

Vertical axis: time steps

Horizontal axis: probability distribution over the vocabulary



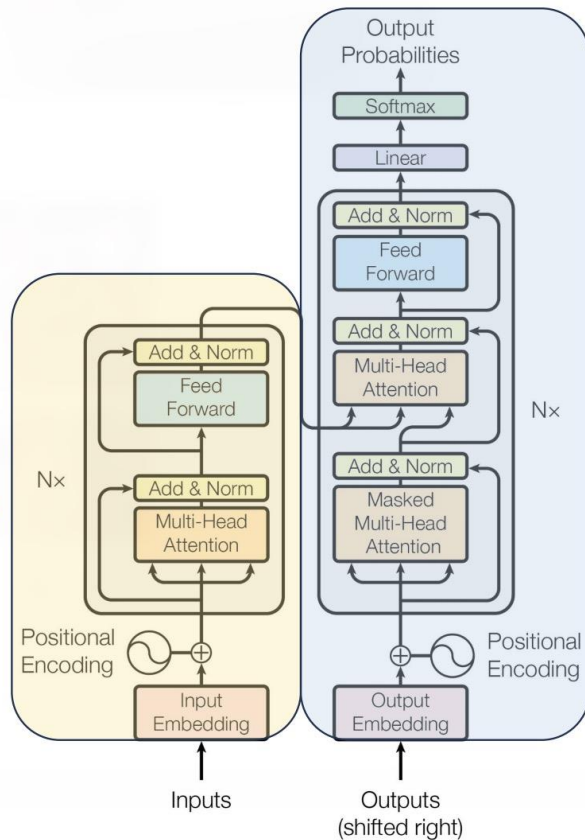
Transformers

Input – input tokens
Output – hidden states

Model can see all timesteps

**Does not usually output tokens, so
no inherent auto-regressivity**

Representation



Input – output tokens and hidden states*

Output – output tokens

**Model can only see previous
timesteps**

**Model is auto-regressive with
previous timesteps' outputs**

Generation

Transformers

Targets

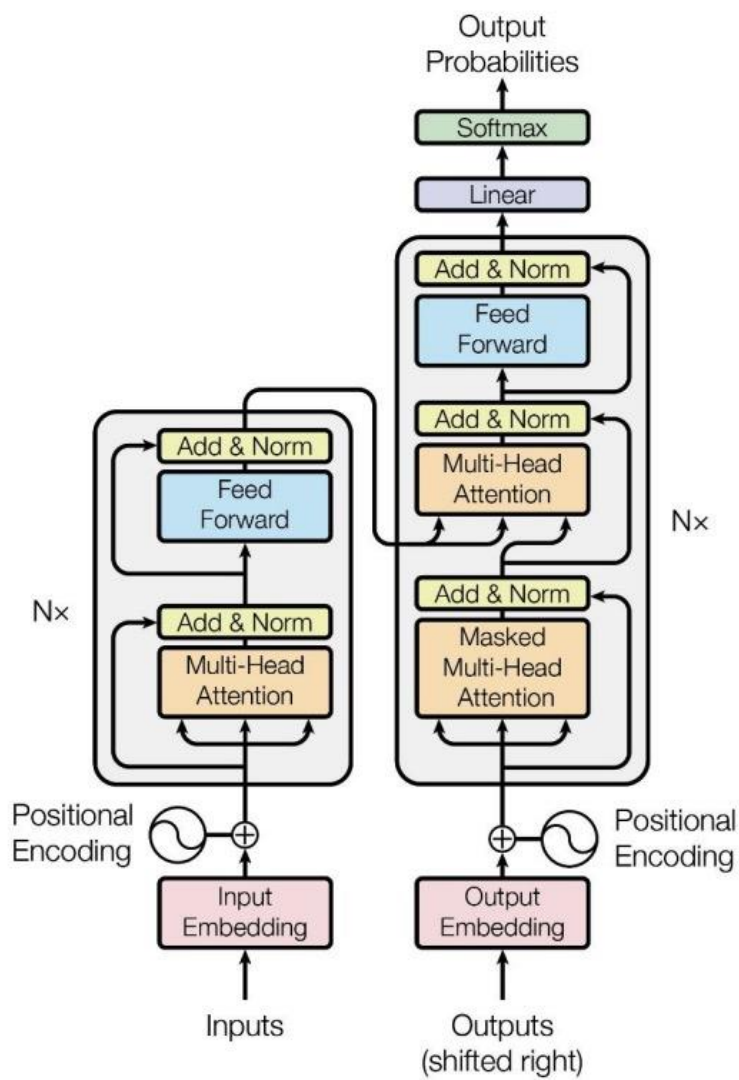
Ich habe einen Apfel
gegessen



Inputs

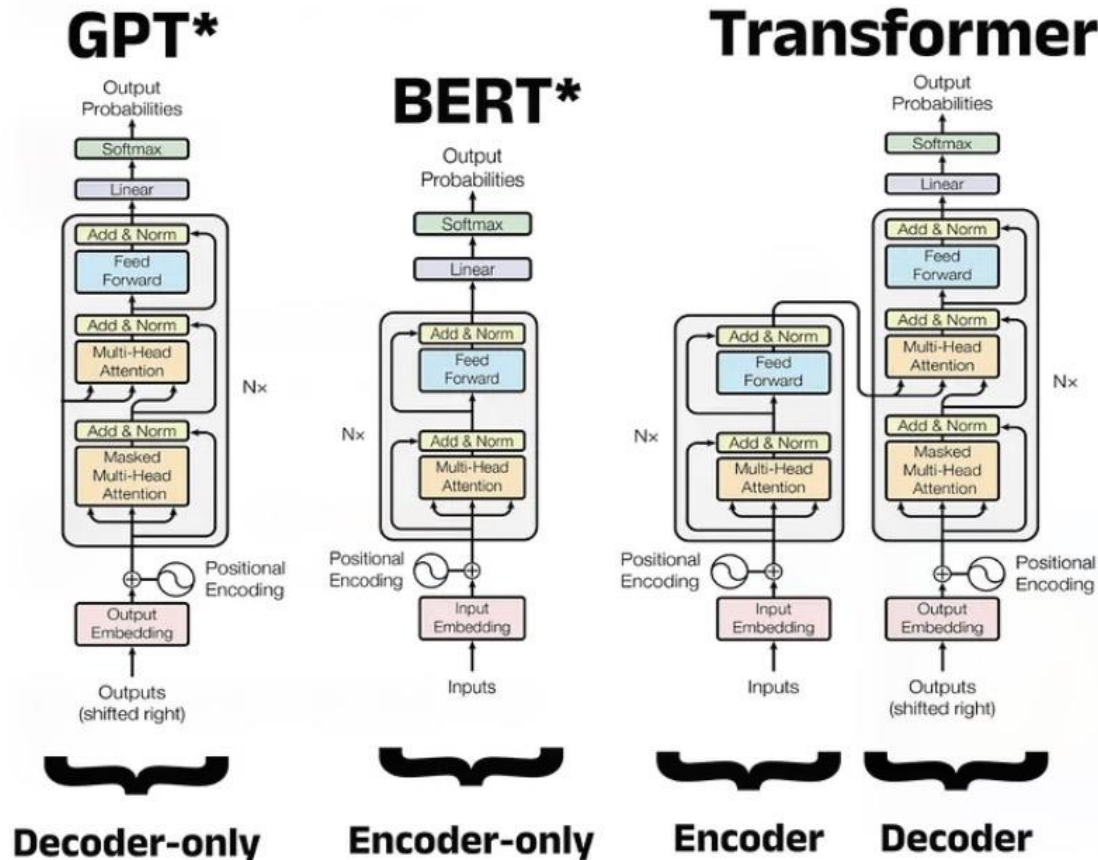
I ate an apple

Machine Translation



GPT - BERT - Transformer

GPT predicts the next word based only on the words before it. This is why GPT is excellent at text generation.

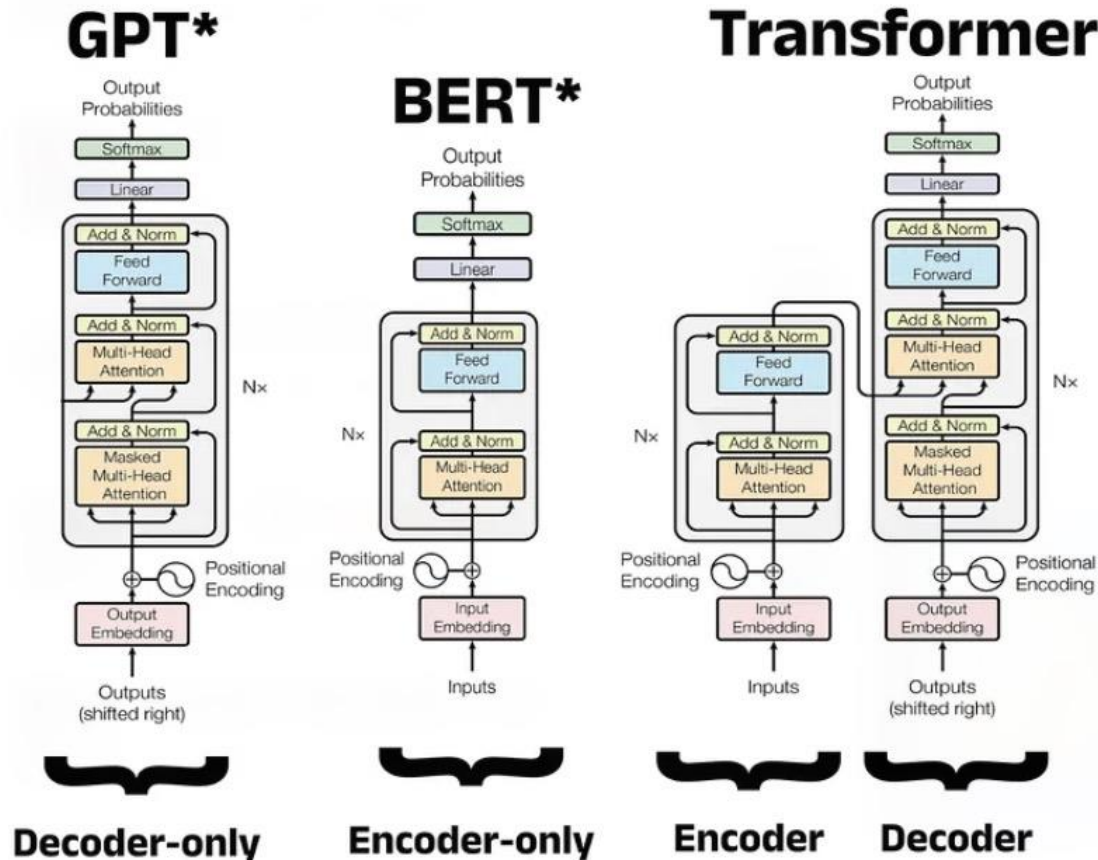


BERT sees the entire sentence at once. It **understands** context in both directions.

Q: Which is better for translation?

GPT - BERT - Transformer

GPT predicts the next word based only on the words before it. This is why GPT is excellent at text **generation**.



BERT sees the entire sentence at once. It **understands** context in both directions.

Q: Which is better for translation?

A: Full Transformer.

GPT - BERT - Transformer

Model	Architecture	Attention Type	Primary Purpose
GPT	Decoder-only	Masked (causal)	Language generation
BERT	Encoder-only	Bidirectional	Understanding tasks
Transformer	Encoder + Decoder	Combined	Translation & seq2seq

ViT (Vision Transformers)



Input image:
e.g. 224x224x3

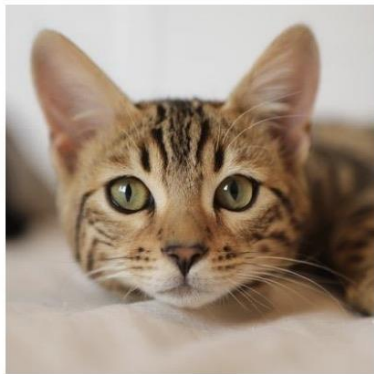


Break into patches
e.g. 16x16x3

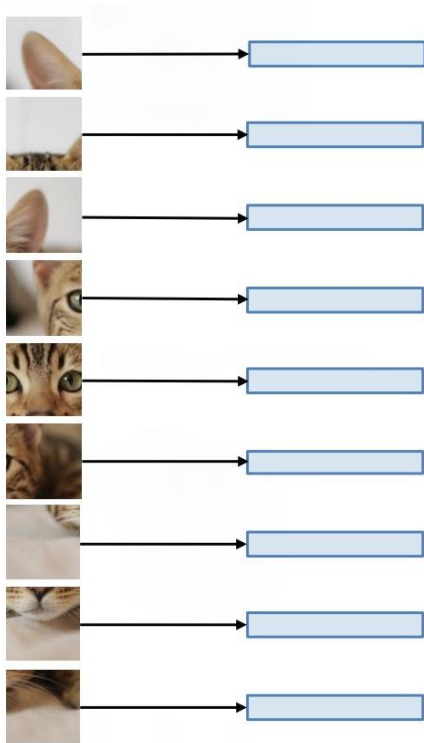
Dosovitskiy et al, "An Image is Worth
16x16 Words: Transformers for Image
Recognition at Scale", ICLR 2021

The following slides are developed based on https://cs231n.stanford.edu/slides/2025/lecture_8.pdf

ViT



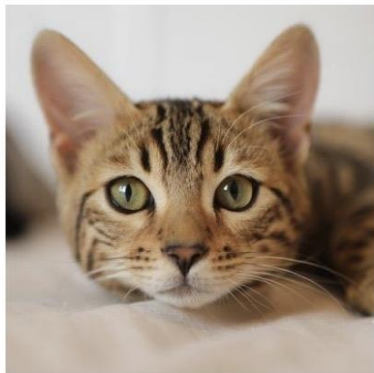
Input image:
e.g. 224x224x3



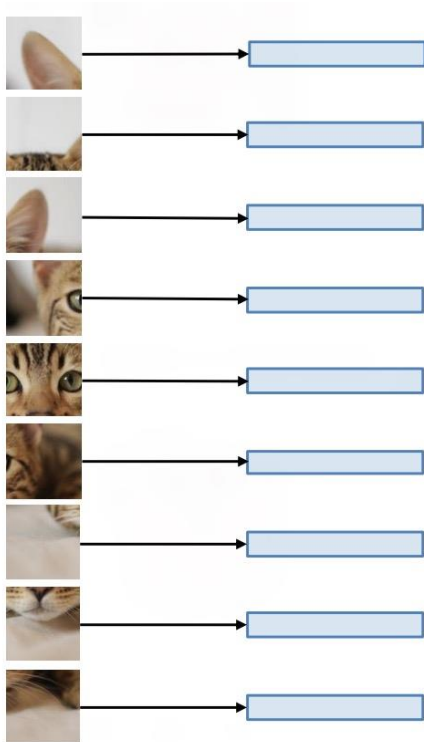
Break into patches
e.g. 16x16x3

Flatten and apply a linear
transform $768 \Rightarrow D$

ViT



Input image:
e.g. 224x224x3

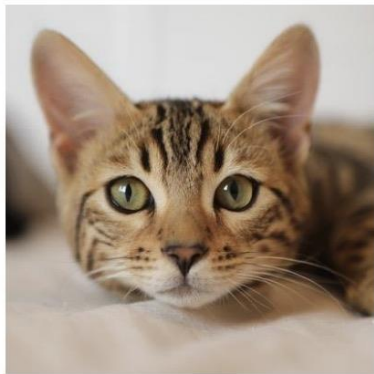


Break into patches
e.g. 16x16x3

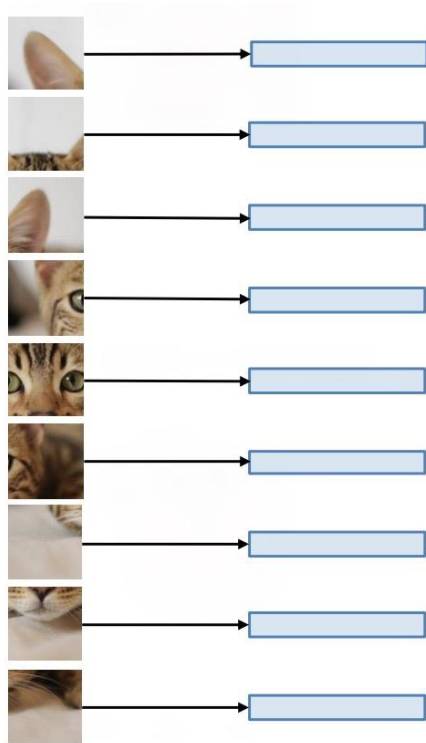
Flatten and apply a linear
transform $768 \Rightarrow D$

Q: Any other way to describe this operation?

ViT



Input image:
e.g. 224x224x3



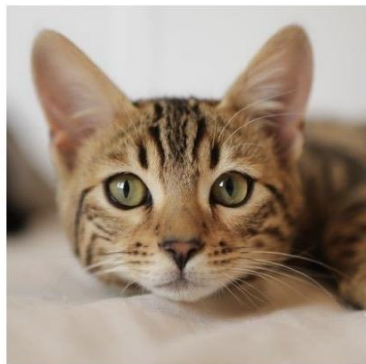
Break into patches
e.g. 16x16x3

Flatten and apply a linear
transform $768 \Rightarrow D$

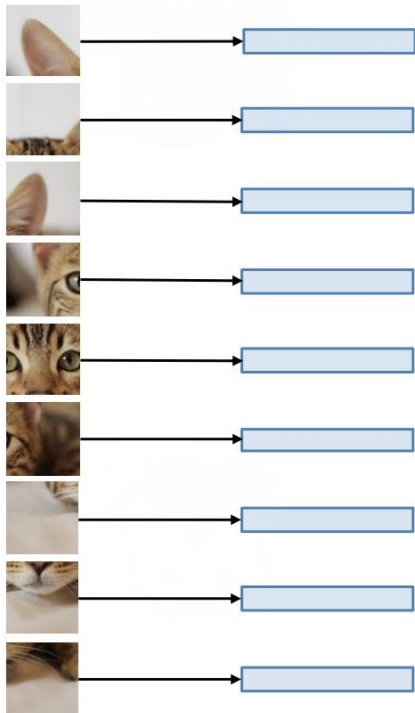
Q: Any other way to describe this operation?

A: 16x16 conv with stride 16, 3 input channels, D output channels.

ViT

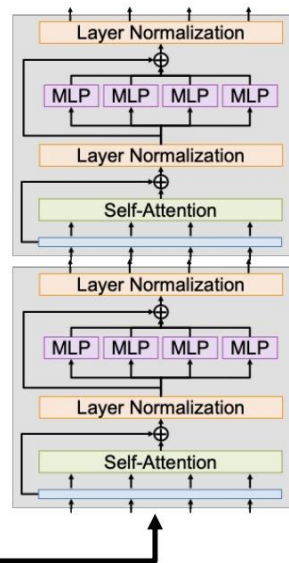


Input image:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

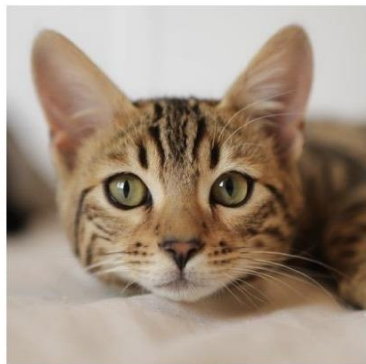
Flatten and apply a linear
transform $768 \Rightarrow D$



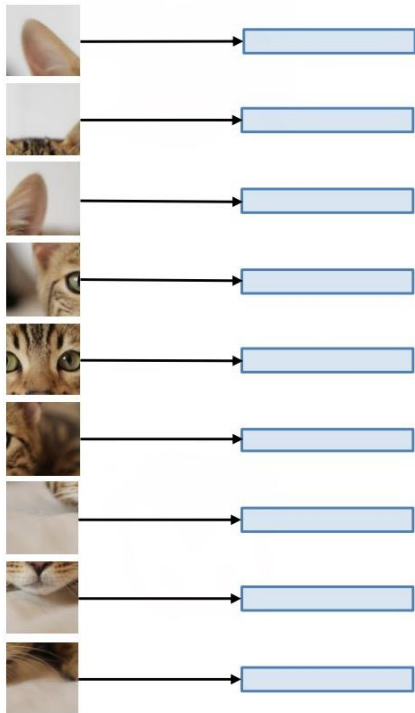
Use **positional encoding** to tell the transformer the 2D position of each patch

D-dim vector per patch
are the input vectors to
the Transformer

ViT

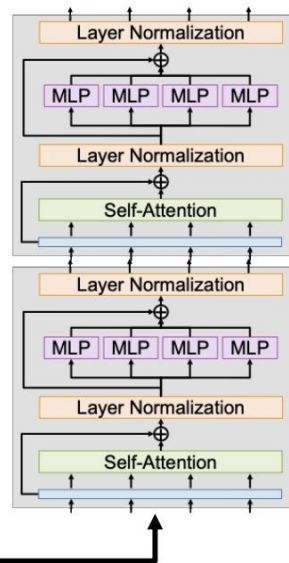


Input image:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

Flatten and apply a linear
transform $768 \Rightarrow D$

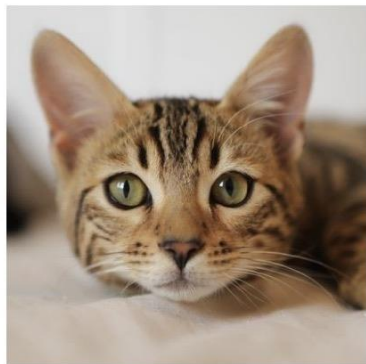


D-dim vector per patch
are the input vectors to
the Transformer

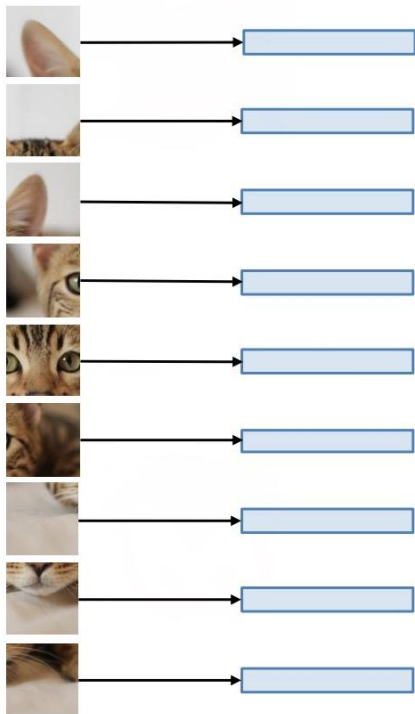
Don't use any masking; each image patch can look at all other image patches

Use **positional encoding** to tell the transformer the 2D position of each patch

ViT

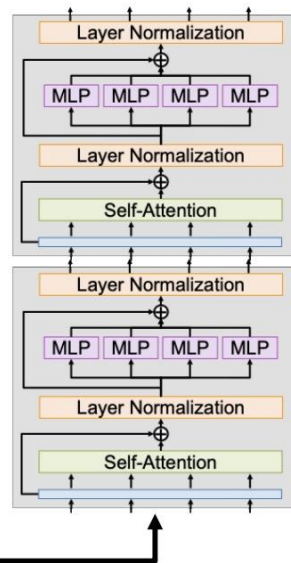


Input image:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

Flatten and apply a linear
transform $768 \Rightarrow D$



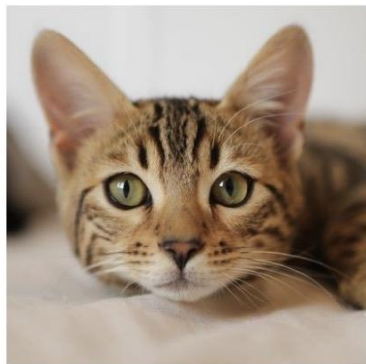
D-dim vector per patch
are the input vectors to
the Transformer

Transformer gives an
output vector per patch

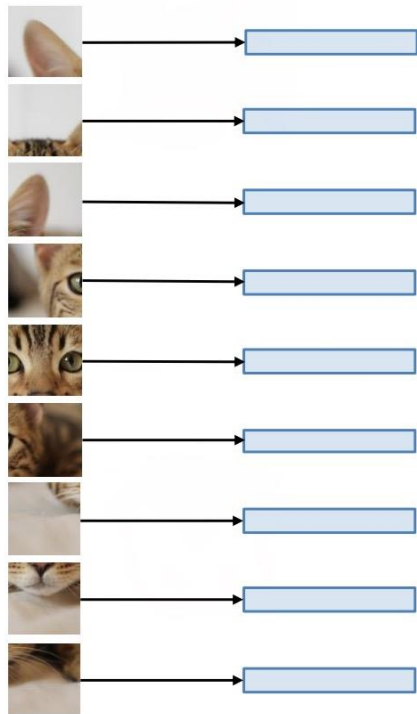
**Don't use any
masking**; each
image patch can look
at all other image
patches

Use **positional
encoding** to tell the
transformer the 2D
position of each
patch

ViT



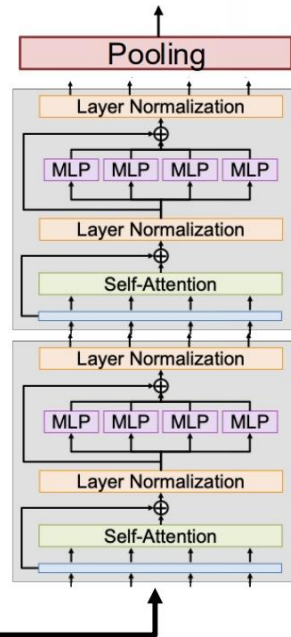
Input image:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

Flatten and apply a linear
transform $768 \Rightarrow D$

Average pool $N \times D$ vectors to $1 \times D$, apply a
linear layer $D \Rightarrow C$ to predict class scores



Transformer gives an
output vector per patch

**Don't use any
masking**; each
image patch can look
at all other image
patches

Use **positional
encoding** to tell the
transformer the 2D
position of each
patch

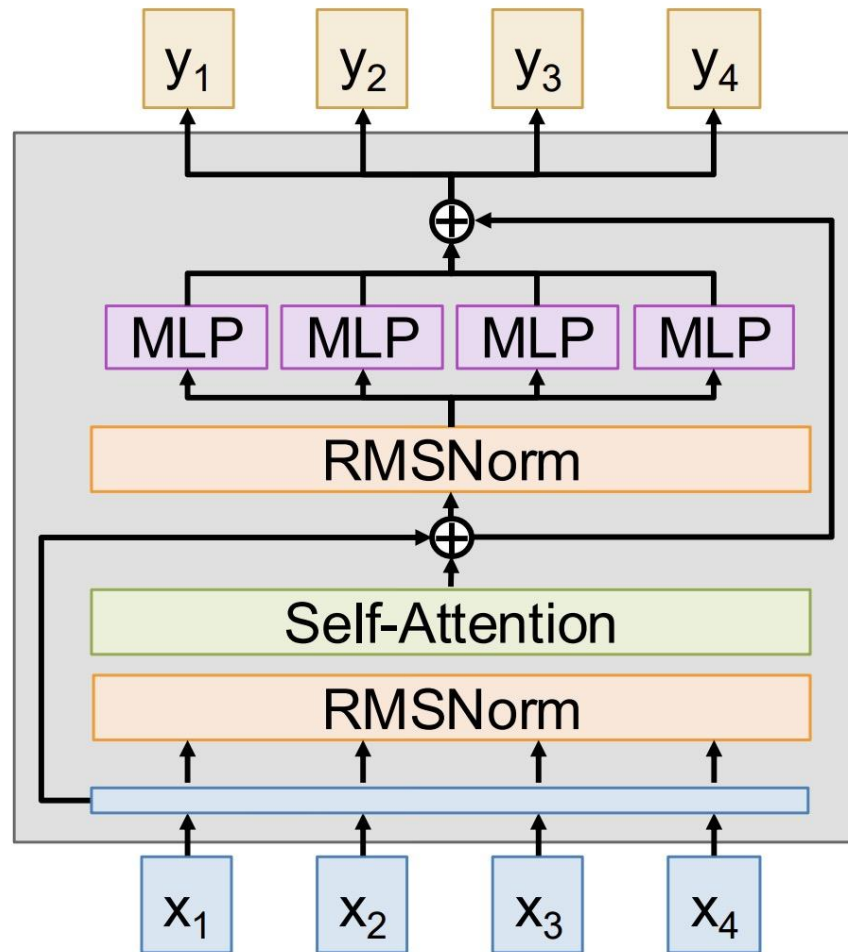
D -dim vector per patch
are the input vectors to
the Transformer

Tweaking Transformers

The Transformer architecture has not changed much since 2017.

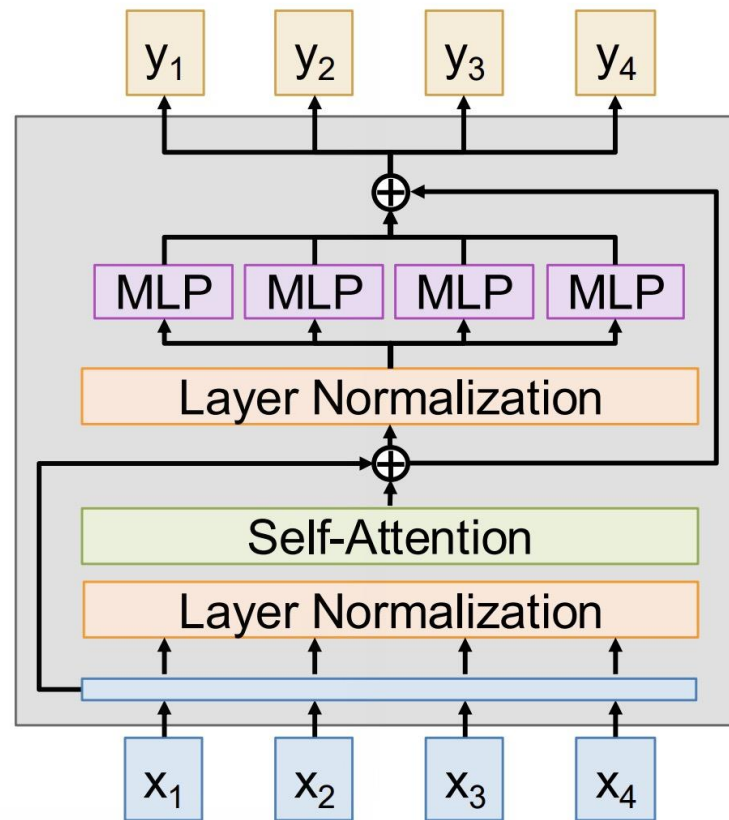
But a few changes have become common:

- **Pre-Norm**: Move normalisation inside residual
- **RMSNorm**: Different normalization layer
- **SwiGLU**: Different MLP architecture
- **Mixture of Experts (MoE)**: Learn E different MLPs, use $A < E$ of them per token.



Pre-Norm Transformer

Layer normalisation is outside the residual connections

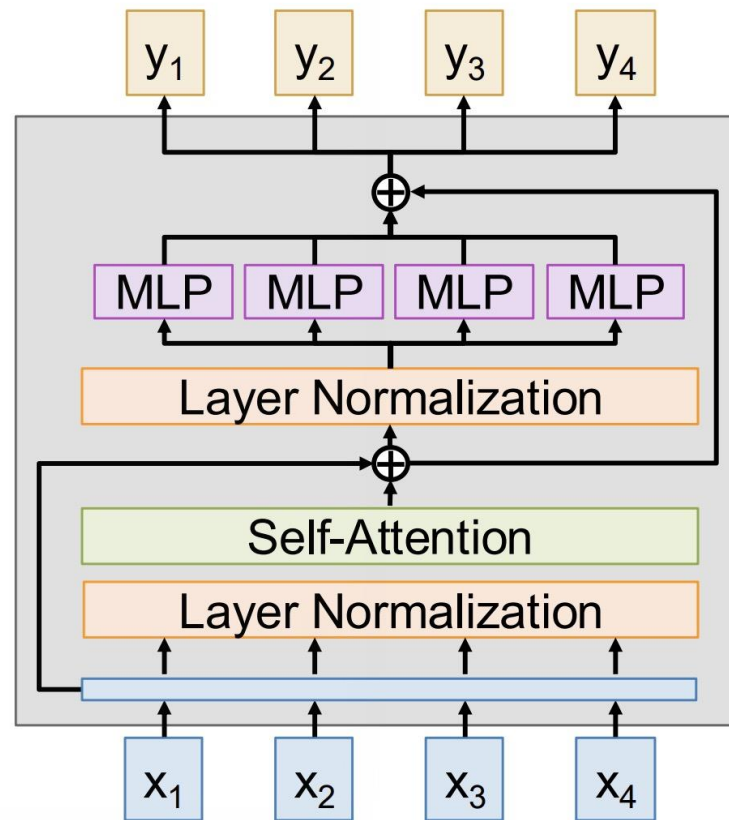


Pre-Norm Transformer

Layer normalisation is outside the residual connections

Kind of weird, the model can't actually learn the identity function

Solution: **Move layer normalisation** before the SelfAttention and MLP, inside the residual connections. Training is more stable.



RMSNorm

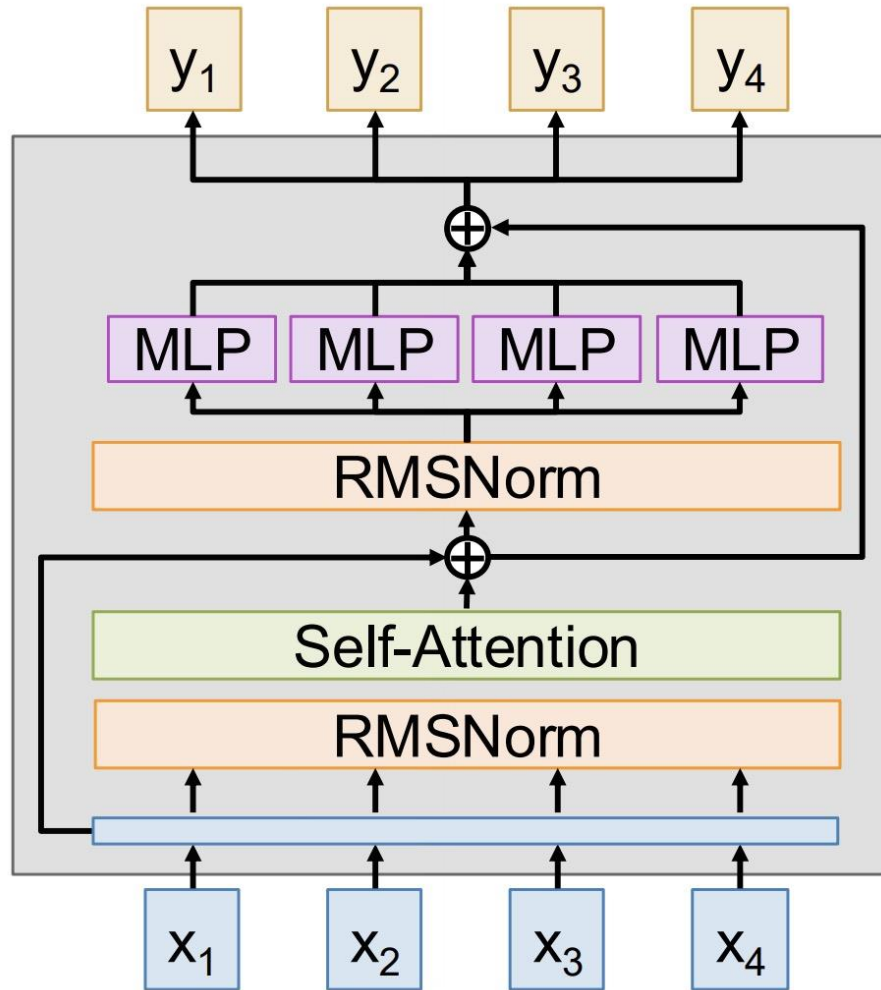
Replace Layer Normalisation with
Root-Mean-Square Normalisation
(RMSNorm)

Input: x [shape D]

Output: y [shape D]

Weight: γ [shape D]

$$y_i = \frac{x_i}{RMS(x)} * \gamma_i$$
$$RMS(x) = \sqrt{\varepsilon + \frac{1}{N} \sum_{i=1}^N x_i^2}$$



SwiGLU MLP

Classic MLP:

Input: $X [N \times D]$

Weights: $W1 [D \times 4D]$ $W2 [4D \times D]$

Output: $Y = \sigma(XW1)W2$

SwiGLU MLP:

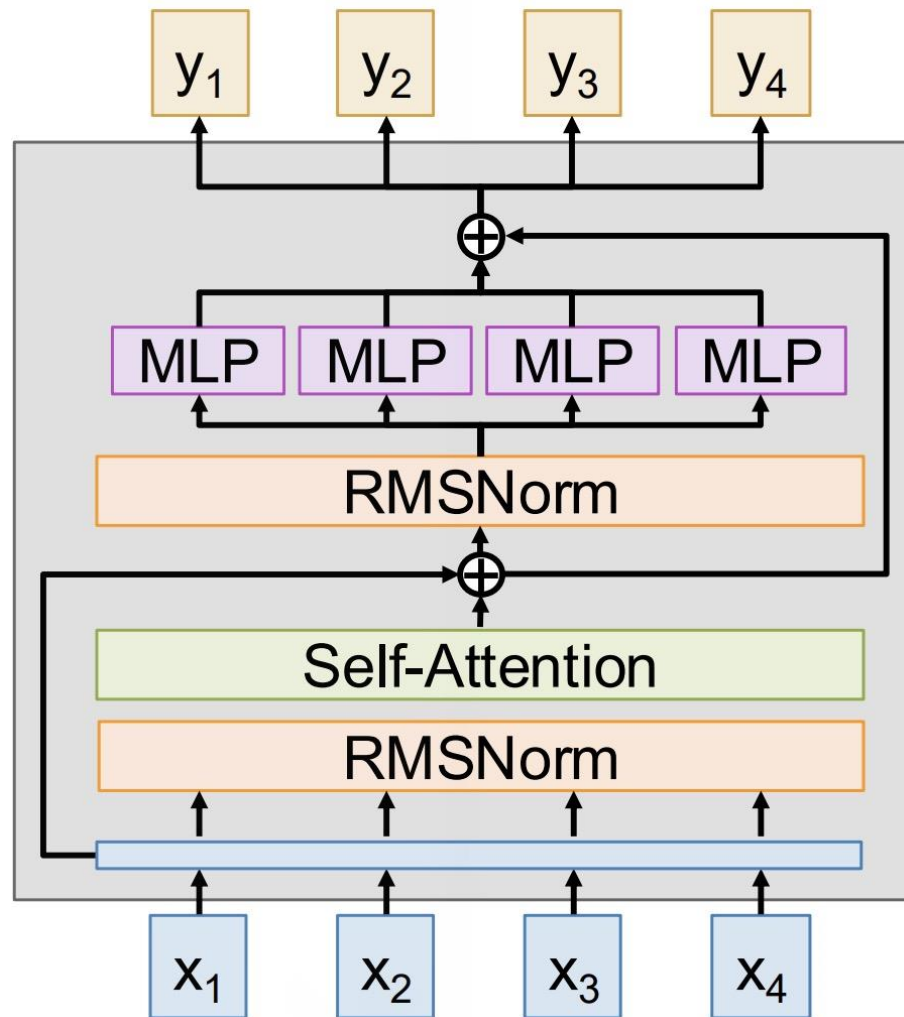
Input: $X [N \times D]$

Weights: $W1, W2 [D \times H]$ $W3 [H \times D]$

Output: $Y = (\sigma(XW1) \odot XW2) W3$

Setting $H = 8D/3$ keeps same total params

No explanation, just better performance.



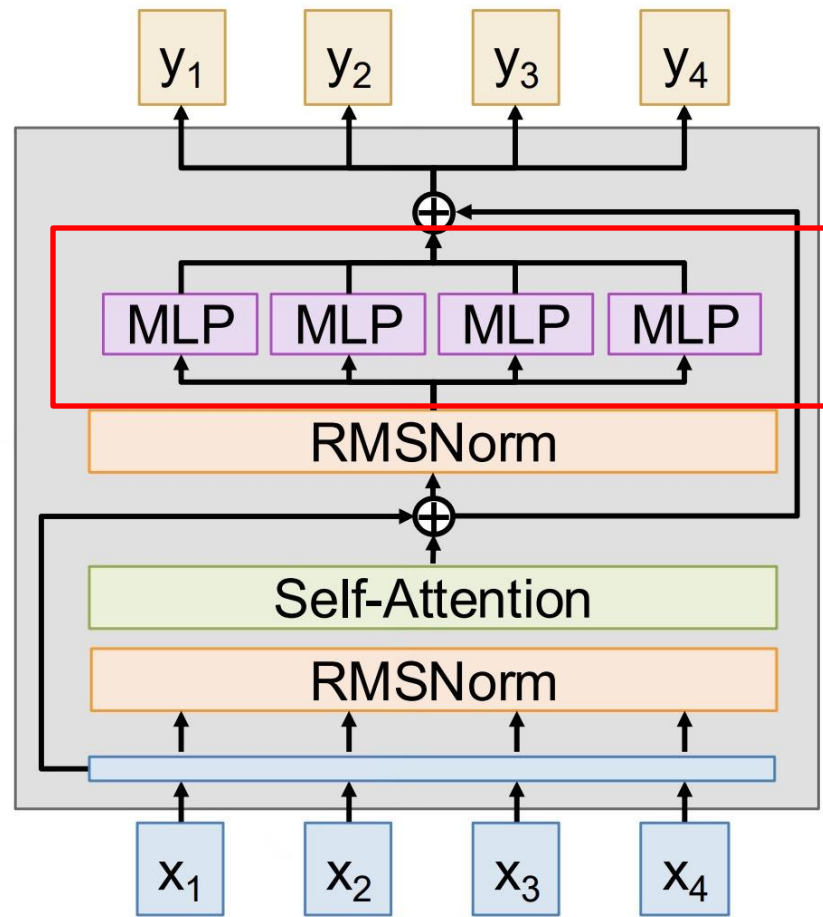
Mixture of Experts (MoE)

Replace a single large feed-forward network with multiple smaller "expert" sub-networks

Computational Efficiency: remaining computationally efficient during inference because they only use a fraction of their total parameters for each input.

Faster Training: By specialising experts and reducing the overall computation per step, MoE models can train faster than dense models of comparable capacity.

Scalability: MoE allows for scaling models to massive sizes.



Summary

Core Architecture

- **Transformer = Attention-based model** replacing recurrence and convolution
- Composed of:
 - **Encoder**: Builds contextual representations of input tokens
 - **Decoder**: Generates output tokens step-by-step

Transformers shift NLP from sequence processing to **global attention-based reasoning**, enabling scalable and powerful language understanding and generation.

Summary

Key Mechanisms

- **Self-Attention**
 - Each token attends to all other tokens
 - Captures long-range dependencies efficiently
- **Multi-Head Attention**
 - Learns multiple representation subspaces in parallel
- **Positional Encoding**
 - Injects order information (since attention is permutation-invariant)
- **Residual Connections + Normalisation**
 - Stabilise training and improve gradient flow

**Next Week: Pre-training,
Fine-tuning, Reasoning,
Challenges, ...**